

# 北京邮电大学

## 《程序设计实验》详细设计文档



题目: 基于 Agent 技术的形式语言与自动机辅助学习游戏《GodSay》

班 级 2022219107

学 号 2022211799 2022211733

2022211740 2022211738

姓 名 王艺栋 王殿云 池瀚 王博远

指导教师 刘瑞芳

2024 年 5 月

# 1、系统概述

## 1.1、系统简介

本项目服务于北京邮电大学《形式语言与自动机》课程教学，以 RPG（角色扮演）游戏的形式带领玩家一步步深入探索神奇的魔法世界，在完成任务与战斗的过程中逐渐学习形式语言与自动机的核心概念与算法。不同于传统游戏基于规则的 NPC 设计与固定的剧情线路，我们的项目通过人物设计与 ChatGPT 接入，为游戏引入了由 ChatGPT 自主控制的 Agent（智能体），为玩家提供了形式语言与自动机的全自动答疑，与更加丰富的交互体验，让玩家在新奇且沉浸的体验中，感受形式语言与自动机知识的美妙。

本项目主要使用 web 开发技术，让玩家可以通过浏览器访问我们的游戏，提供全平台浏览器支持（Windows，MacOS，Android 与 IOS），跨越了平台壁垒，为更多玩家提供服务。

项目目前已由 GitHub Page 托管发布，欢迎您来体验！

游戏：

<https://wangdianyun.github.io/GodSay/>

展示：

<https://gamma.app/docs/GodSay-tv028highfk56sk>

本系统设计的主要创新点有：

### 1. 寓教于乐的教学模式

不同于传统教学辅助工具的枯燥乏味，本系统引入 ARPG 游戏模式，帮助学生在游戏中学习新知识，在增强学生学习自主性的同时降低了知识学习过程中的枯燥乏味。

### 2.LLM 赋能 Agent 自主答疑

大语言模型自主决策，让游戏中的非玩家角色拥有自主探索环境、不设限回复的能力。突破了传统游戏中 NPC 交互死板的局限，实现了游戏角色的智能交互，增加了游戏的沉浸感。并且采用模块化设计，可以灵活地定制和扩展角色的能力，便于维护和管理。

### 3. 内嵌式形式语言与自动机解密插件

知识只有在应用中才能掌握扎实，通过开发形式语言与自动机解密插件，我们为玩家提供了在游戏中交互式解题的方式，并在游戏中加入了大量解谜元素。谜题即形式语言与自动机的题目。玩家可以通过构造自己的 DFA、NFA、CFG、REG 和 PDA 来获得武器、解除禁制、学习魔法。

在游戏中，我们为玩家设计了一个自动机可视化构造模块，它主要负责给用

户提供一个直观友好的自动机状态和转换关系编辑页面，帮助用户直观理解自动机的运行和状态转移，并在解决谜题的同时学习知识。它的主要功能包括：绘制状态图、编辑状态和转换、设置初始和终止状态、输入文法、测试自动机接受的字符串。

1.2、术语表

定义系统或产品中涉及的重要术语，为读者在阅读文档时提供必要的参考信息。

| 序号 | 术语或缩略语 | 说明性定义                                                  |
|----|--------|--------------------------------------------------------|
| 1  | LLM    | Large Language Model<br>大语言模型，文本类生成式人工智能               |
| 2  | Agent  | 智能体，指能够理解环境并做出响应的个体，本文中主要指由 LLM 控制的，可以与玩家交互的非玩家角色（NPC） |
| 3  | NPC    | Non Player Character，非玩家角色，指游戏中不由玩家控制的角色               |
| 4  | CDN    | 内容分发网络，一种通过分发式请求缓解单台服务器负载的服务端方法，能够处理用户的高并发请求           |

1.3、系统运行环境

我们的系统使用服务器托管，用户端使用 HTTP 或 HTTPS 协议向服务器发起请求并拉取游戏内容，服务器返回游戏的逻辑与数据，具体信息如下：

1.3.1 服务器硬件平台

系统的服务器端运行在支持 Ubuntu（≥20.04）或 CentOS（≥7-1511）系统的服务器硬件平台（AMD64、Intel 64 与 Intel x86 为主）上，服务器用于托管 Web 页面与资源，并供客户端访问。出于性能考虑，每台服务器硬件平台至少需要拥有 64GB 的内存与 128GB 的储存空间，并通过 CDN 处理高并发请求。出于成本考虑，本项目选择 GitHub 的页面托管服务来分发页面，由 GitHub 自身的服务器和基础设施托管。

GitHub 的服务器位于多个数据中心，具有高可用性、稳定性和性能。并且 GitHub Pages 使用 CDN 来加速内容交付，提供全球范围内快速访问网站的能力，符合我们的要求。

### 1.3.2 服务器操作系统

出于稳定性和开源特性，系统的服务器端主要采用 Linux 操作系统，考虑性能和特性，具体主要包括 Ubuntu (≥20.04) 或 CentOS (≥7-1511)，为用户提供可靠安全的服务。

### 1.3.3 客户端硬件平台

客户端硬件平台要求主要集中于图形处理与网络请求，本游戏采用 2D 像素风设计，无 3D 渲染需求，因此客户端的硬件需要能荷载 60 帧 720P 的简单图形处理，并同步处理交互与音频等信息，要求硬件具有 4G 及以上的内存。网络请求上，出于 ChatGPT 的交互需求，要求客户端能够正常与 <https://api.openai.com/v1/chat/completions> 完成通信。

### 1.3.4 客户端操作系统

客户端操作系统无特殊限制，仅需包含图形化界面支持。由于本项目用 web 页面交互，并针对包括 Chrome、Edge、Firefox、Safari 在内的浏览器优化，因此理论上用户可以使用 Windows、MacOS、Android、IOS、Ubuntu 在内的系统访问我们的系统。

### 1.3.6 数据库系统

本项目使用了关系型数据库 MySQL 完成用户信息的查询与修改，并能保存不同用户的游戏进度，帮助用户循序渐进地学习知识。

### 1.3.7 编程平台

前端开发主要使用 HTML、CSS、JavaScript 进行，构建网页结构、样式和交互，并且使用 RPGmakerMZ web 游戏开发框架完成形式语言与自动机答题页面的设计，使用 Node.js 完成项目的管理与测试开发。

后端开发较为轻量级，因此主要使用 python 结合 MySQL 进行开发，使用 mysql-connector-python 框架完成功能设计与开发。

### 1.3.8 网络协议

本项目中主要使用到的网络协议包括 HTTP 和 HTTPS 协议，以在服务器端与客户端之间通信。

网络通信操作在本项目中主要负责：

1. 从服务器拉取游戏页面与资源，供玩家游玩；
2. 从服务器拉取形式语言与自动机解题页并嵌入游戏页面；
3. 与 <https://api.openai.com/v1/chat/completions> 通信，调取 ChatGPT 的 API 以完成 Agent 的构建；

#### 1.4、开发环境

IDE：

1. IntelliJ IDEA 2024.1 (Ultimate Edition)
2. PyCharm 2024.1.1 (Professional Edition)
3. RPG Maker MZ

开发语言：

1. HTML(Living Standard)
2. JavaScript (ES2023)
3. Python (3.11.8)
4. SQL(适用 8.0.36 MySQL Community Server)

包管理：

1. Node.js (18.20.0)
2. Conda (23.7.4)

项目管理工具：

1. Git(2.42.0.windows.2)

库：

1. Python 库
  - a. mysql-connector-python: 8.4.0
  - b. flask: 3.0.3
2. Node 包

- a. bootstrap: 4.4.1,
- b. cfgrammar-tool: 1.0.0,
- c. d3: 5.15.0,
- d. react: 16.12.0,
- e. react-bootstrap: 1.0.0-beta.16,
- f. react-contextmenu: 2.13.0,
- g. react-d3-graph: 2.3.0,
- h. react-dom: 16.12.0,
- i. react-files: 2.4.8,
- j. react-scripts: 3.3.1,
- k. react-sidebar: 3.0.2,
- l. react-tooltip: 4.2.9,
- m. reactjs-popup: 1.5.0,
- n. uuid: 8.0.0,
- o. vis-network: 7.3.3

测试环境

- 1. 服务器 Ubuntu20.04
- 2. 客户端 Windows 11 家庭中文版：  
Google Chrome 125.0.6422.60（正式版本）（64 位）  
Microsoft Edge 125.0.2535.51 (正式版本) (64 位)  
Firefox Browser 126.0 (64 位)
- 3. 客户端 IOS 17.4.1  
Safari 4.0

1.5、关键知识点

列举本项目涉及到的知识点及其关系，比如粗粒度知识点“包含”细粒度知识点，可以是对比关系、顺序关系等等。

知识点范围包括：计算机应用、人工智能、信息与通信等领域。

|    |       |    |       |
|----|-------|----|-------|
| 序号 | 知识点 1 | 关系 | 知识点 2 |
|----|-------|----|-------|

|    |          |    |                            |
|----|----------|----|----------------------------|
| 1  | 形式语言与自动机 | 包含 | DFA 的构造与识别                 |
| 2  | 形式语言与自动机 | 包含 | NFA 的构造与识别                 |
| 3  | 形式语言与自动机 | 包含 | CFG 的构造与识别                 |
| 4  | 形式语言与自动机 | 包含 | PDA 的构造与识别                 |
| 5  | 形式语言与自动机 | 包含 | REG 的构造与识别                 |
| 6  | 形式语言与自动机 | 包含 | 语言、文法基础知识                  |
| 7  | 前端开发     | 包含 | HTML 语言                    |
| 8  | 前端开发     | 包含 | CSS 语言                     |
| 9  | 前端开发     | 包含 | JavaScript 语言              |
| 10 | 前端开发     | 包含 | React 前端开发框架               |
| 11 | 前端开发     | 包含 | Node.js 使用方法               |
| 12 | 前端开发     | 包含 | Webpack 打包方法               |
| 13 | 后端开发     | 包含 | Python 语言                  |
| 14 | 后端开发     | 包含 | MySQL 数据库                  |
| 15 | 后端开发     | 包含 | SQL 语言                     |
| 16 | 后端开发     | 包含 | mysql-connector-python 库使用 |
| 17 | 后端开发     | 包含 | Flask 库使用                  |

|    |           |    |                |
|----|-----------|----|----------------|
| 18 | 网络通信协议    | 包含 | HTTP           |
| 19 | 网络通信协议    | 包含 | HTTPS          |
| 20 | 网络通信协议    | 包含 | GET 方法         |
| 21 | API 接口协议  | 包含 | ChatGPT API 接口 |
| 22 | Prompt 工程 | 包含 | 世界观 Prompt 设计  |
| 23 | Prompt 工程 | 包含 | 角色人格 Prompt 设计 |

## 1.6、知识链条



## 2、数据结构说明

### 2.1、常量

#### • 2.1.1 APRG 插件

本处无需要强调的常量



- 2.1.2 游戏主体

常量主要是游戏配置数据，具体数据文件名称及其所在目录如下：

- DATA\_FILE\_NAME: 字符串，表示游戏数据文件的名称。
- DATA\_DIRECTORY: 字符串，表示游戏数据文件所在的目录路径。
- PluginManager 类用于管理插件，包括加载、设置参数、注册命令等功能。
- Actors.json: 角色数据文件，位于 data/Actors.json。
- Classes.json: 职业数据文件，位于 data/Classes.json。
- Skills.json: 技能数据文件，位于 data/Skills.json。
- Items.json: 物品数据文件，位于 data/Items.json。
- Weapons.json: 武器数据文件，位于 data/Weapons.json。
- Armors.json: 护甲数据文件，位于 data/Armors.json。
- Enemies.json: 敌人数据文件，位于 data/Enemies.json。
- Troops.json: 部队数据文件，位于 data/Troops.json。
- States.json: 状态数据文件，位于 data/States.json。
- Animations.json: 动画数据文件，位于 data/Animations.json。
- Tilesets.json: 地图块数据文件，位于 data/Tilesets.json。
- CommonEvents.json: 公共事件数据文件，位于 data/CommonEvents.json。
- System.json: 系统设置数据文件，位于 data/System.json。
- MapInfos.json: 地图信息数据文件，位于 data/MapInfos.json。
- 在项目的 \data 目录下，这些常量文件存储了游戏中各种数据和文本信息，供游戏运行时使用：
- Game\_System: 存储游戏系统相关的数据，如战斗 BGM、背景音乐等。
- Game\_Party: 存储玩家队伍相关的数据。
- Game\_Troop: 存储敌对队伍相关的数据。
- TextManager: 存储游戏中使用的文本信息，如道具名称、技能名称等。

- 2.1.3 玩家管理

后端：

- `db_config`: 用于存储数据库配置的字典常量。

Python

```
db_config = {'host': 'localhost', 'user': 'root', 'password':  
'040803', 'database': 'user_management'}
```

前端:

- `login-container`, `register-container`: 用于表示登录和注册表单的容器元素 ID 常量。

HTML

```
<div class="container" id="login-container"...</div><div  
class="container" id="register-container" style="display:  
none;"...</div>
```

- 2.1.4 FLAP 形式语言与自动机解密插件
  - a. `CURRENT_MACHINE` 用于区分不同类型的自动机如 DFA、NFA、CFG、TM、PDA、REG。
  - b. `error_image`, `success_image`, `idle_svg`, `add_perfect`: 分别存储各种状态和按钮的图片资源。
  - c. `types`, `parser`, `generator`: 从 `cfgrammar-tool` 库导入的相关功能, 用于处理文法生成、解析等。
  - d. `rflapsvg`: 导入的 SVG 文件, 用作 Logo。
  - e. `accept_bar`, `add_bar`, `points_bar`, `reject_bar`, `transition_bar`, `blank_svg_bar`, `passive_bar`, `remove_bar`, `add_perfect`: 这些图片资源用于在界面上表示各种状态和功能。
  - f. `image_collection`: 包含错误、空闲、成功状态的图片路径数组。
  - g. `error_object`: 错误对象, 用于跟踪各种错误状态。
  - h. `packet_to_misha_the_microsoft_engineer`: 包含 PDA、字母表、起始状态、状态、转换函数、确定性、输入字符串等信息的对象。
- 2.1.5 ChatGPT-Agent 插件
  - `parameters`:

JavaScript

```
const parameters =  
PluginManager.parameters('ChatGPTIntegration');
```

- worldSetting:

```
JavaScript
const worldSetting = parameters['worldSetting'];
```

- apiKey:

```
JavaScript
const apiKey = parameters['apiKey'];
```

- modelName:

```
JavaScript
const modelName = "gpt-3.5-turbo";
```

- url:

```
JavaScript
let url = "https://api.openai.com/v1/chat/completions";
```

## 2.2、变量

- 2.2.1 ARPG 插件

- 主要变量

- simpleExported:

用于存储导出的模块。

```
var simpleExported;
if (typeof simpleExported === "undefined") {
    simpleExported = {};
}
```

- ARPG\_CorePluginParams:

插件参数，用于存储解析后的插件配置。

```
var ARPG_CorePluginParams =
PluginParamsParser.parse(PluginManager.parameters(ARPG_CorePlug
inName));
```

- \_\_tmp\_\_require:

临时保存 require 函数。

```
var __tmp__require = require;
```

- ARPG\_CorePluginName:

插件名称，基于当前脚本文件名自动获取。

- ARPG\_CorePluginParams:

插件参数，通过插件参数解析器解析。

- 2.2.2 游戏主体

值得强调的变量主要在 Bitmap, Graphics 和 Sprite 类中:

- Graphics 类变量

在游戏的图形处理和界面反馈中起着核心作用，用于管理和更新游戏的视觉表现和性能监控。

- Graphics.\_width 和 Graphics.\_height: 存储游戏屏幕的宽度和高度。
- Graphics.\_realScale: 根据设备和设置调整的实际显示比例。
- Graphics.\_canvas: 指向游戏渲染的 HTML canvas 元素的引用。
- Graphics.\_app: 指向 PIXI 应用实例的引用，用于游戏的图形渲染。
- Graphics.\_errorPrinter: 用于显示错误信息的 HTML 元素。
- Graphics.\_loadingSpinner: 加载时显示的旋转动画 HTML 元素。
- Graphics.\_fpsCounter: 用于追踪和显示帧率 (FPS) 的对象。

- Bitmap 类变量:

- \_canvas: 存储绘图的 HTML Canvas 元素。
- \_context: 存储 Canvas 的 2D 渲染上下文。
- \_baseTexture: PIXI 的基础纹理，用于图像处理。
- \_image: HTML 图像元素，用于加载和显示图像。
- \_url: 图像资源的 URL 地址。
- \_loadingState: 表示图像的加载状态 ("none", "loading", "loaded", "error") 。

- Sprite 类变量:

主要用于存储状态信息或对象的配置参数:

- `_bitmap`: 关联的 `Bitmap` 对象，用于渲染图像。
- `_frame`: 定义精灵图的显示区域。
- `_hue`: 色调调整值。
- `_blendColor` 和 `_colorTone`: 用于调整精灵图的颜色和混合模式。
- `Tilemap` 类变量:
  - `_width` 和 `_height`: 存储瓦片地图的宽度和高度。
  - `_margin`: 地图边缘留白的大小，用于处理滚动时的视觉效果。
  - `_mapWidth` 和 `_mapHeight`: 地图的实际宽度和高度（以瓦片为单位）。
  - `_mapData`: 存储地图数据，通常是一维或二维数组。
  - `_bitmaps`: 存储用于瓦片绘制的位图数组。
  - `tileWidth` 和 `tileHeight`: 单个瓦片的宽度和高度。
  - `origin`: 用于滚动操作的原点位置，是一个 `Point` 对象。
  - `flags`: 存储瓦片的特性标志，例如是否可通过等。
- `animationCount`:
 

用于处理动画效果的计数器类

  - `_mousePressed`: 鼠标是否按下。
  - `_screenPressed`: 屏幕是否按下。
  - `_pressedTime`: 按下的时间。
  - `_clicked`: 是否点击。
  - `_newState`: 新的输入状态。
  - `_currentState`: 当前输入状态。
  - `_x`: 最新触摸事件在画布上的 x 坐标。
  - `_y`: 最新触摸事件在画布上的 y 坐标。
  - `_triggerX`: 触发事件的 x 坐标。
  - `_triggerY`: 触发事件的 y 坐标。
  - `_moved`: 鼠标或触摸是否移动。
  - `_date`: 最后一次输入的时间。
- 全局数据变量:
  - `$dataActors`: 角色数据变量。

- \$dataClasses: 职业数据变量。
- \$dataSkills: 技能数据变量。
- \$dataItems: 物品数据变量。
- \$dataWeapons: 武器数据变量。
- \$dataArmors: 护甲数据变量。
- \$dataEnemies: 敌人数据变量。
- \$dataTroops: 部队数据变量。
- \$dataStates: 状态数据变量。
- \$dataAnimations: 动画数据变量。
- \$dataTilesets: 地图块数据变量。
- \$dataCommonEvents: 公共事件数据变量。
- \$dataSystem: 系统设置数据变量。
- \$dataMapInfos: 地图信息数据变量。
- 游戏对象变量
  - \$gameTemp: 游戏临时变量对象。
  - \$gameSystem: 游戏系统对象。
  - \$gameScreen: 游戏画面对象。
  - \$gameTimer: 游戏计时器对象。
  - \$gameMessage: 游戏消息对象。
  - \$gameSwitches: 游戏开关对象。
  - \$gameVariables: 游戏变量对象。
  - \$gameSelfSwitches: 游戏自开关对象。
  - \$gameActors: 游戏角色对象。
  - \$gameParty: 游戏队伍对象。
  - \$gameTroop: 游戏敌群对象。
  - \$gameMap: 游戏地图对象。
  - \$gamePlayer: 游戏玩家对象。
  - \$testEvent: 测试事件对象。
- 数据管理器变量

- `DataManager._globalInfo`: 全局信息数据变量。
- `DataManager._errors`: 数据加载错误列表。
- `DataManager._databaseFiles`: 数据库文件列表。
- `_scripts`: 存储已加载的插件脚本的文件名列表。
- `_errorUrls`: 存储加载插件脚本时出错的 URL 列表。
- `_parameters`: 存储插件参数的对象，键为插件名，值为参数对象。
- `_commands`: 存储注册的命令的对象，键为插件名和命令名的组合，值为对应的函数。

### • 2.2.3 玩家管理

后端：

- `data`: 存储客户端传递的 JSON 数据。
- `username, password, email`: 从 `data` 中提取的用户输入数据。
- `password_hash`: 用于存储加密后的密码。
- `connection, cursor`: 用于数据库连接和操作的变量。
- `user`: 查询数据库后存储用户信息的变量。

前端：

- `username, password, email`: 从表单中获取的用户输入数据。
- `errorMessage`: 用于显示错误信息的 HTML 元素。

```
JavaScript
const username = document.getElementById('login-username').value;
const password = document.getElementById('login-password').value;
const errorMessage = document.getElementById('login-error-message');
```

### • 2.2.4 FLAP 形式语言与自动机解密

a. `master_context` 变量存储关于自动机的全局状态，如图形对象、文法规则和是否为下推自动机的标志。

- `graphObj`: 用于存储自动机图形的参数和对象。
- `grammar_obj`: 用于存储上下文无关文法（CFG）的规则。
- `pushdown`: 布尔值，表示是否为下推自动机（PDA）。

- b. `CFG_Visual_Context_Index`: 用于追踪当前语法规则的索引。
- c. `bool_first_mount`: 标志位, 用于确定是否是组件的首次挂载。
- d. `grammar_table_text`: 存储语法表格中的文本内容。
- e. `image_collection`: 存储表示各种状态的图像数组。
- f. `row_entry_array`、`definition_entry_array`: 分别存储行输入和定义输入的状态数组。
- g. `UIN_input`, `displayWarning`, `warning_display`, `exportModal`: 布尔型状态, 用于控制 UI 中的不同显示和模态框。
- h. `height`: 窗口高度, 用于设置图形界面的尺寸。
- i. `img_bar_status_did_mount`: 标志变量, 用于跟踪图片条是否已加载。
- j. `nodesDS` 和 `edgesDS`: `vis.DataSet` 对象, 用于存储节点和边的数据。
- k. `in_initial_mode`, `in_accepting_mode`, `delete_lock`, `edgeDeletion`: 状态标志, 用于控制不同的用户交互模式。
- l. `graph`: 包含 `nodes` 和 `edges` 的对象, 用于图的渲染和操作。
- m. `master_context`: 通过 `useContext` 获取的上下文, 共享全局状态。
- n. `network`: `vis.Network` 实例, 用于渲染和管理网络图。
- o. `isExport`: 布尔值, 表示是否为导出操作。
- p. `readImportTxt`: 读取的导入文本内容。
- q. `testID`: 测试 ID。
- r. `master_context`: 上下文对象, 包含全局状态。
- s. `user_input_row_collection`: 用户输入的行集合。
- t. `UIN_input`: 布尔值, 表示是否显示 UIN 输入。
- u. `warning_display`: 布尔值, 表示是否显示警告。
- v. `row_entry_array`: 行条目数组, 用于渲染输入行。
- w. `textOrDownload`: 对象, 表示是文本显示还是下载。

- ChatGPT-Agent 插件

- `dialogHistory`:

```
JavaScript
const dialogHistory = [];
```

- `plot`:



```
JavaScript
let plot = ''
```

## 2.3、数据结构

- 2.3.1 ARPG 战斗
  - PluginParamsParser:  
插件参数解析器类。
  - Degree:  
表示角度的类。
  - ARPG\_Utils:  
实用工具类。
  - HitBox:  
碰撞盒类。
  - ComponentRunner:  
组件运行管理类。
  - Component:  
组件基类。
  - HitChecker:  
碰撞检查类。
  - ErrorManager:  
错误管理类。
  - 功能说明
    - PluginParamsParser:  
用于解析插件参数，支持多种数据类型的转换。
    - Degree:  
处理方向和角度的转换。
    - ARPG\_Utils:  
提供各种实用函数，例如查找角色、计算距离等。
    - HitBox:

管理角色的碰撞盒，处理碰撞检测。

- **ComponentRunner:**  
管理和运行组件的更新。
- **Component:**  
组件基类，提供生命周期管理。
- **HitChecker:**  
负责管理和检查碰撞的状态和行为。
- **ErrorManager:**  
提供错误处理和错误消息生成。

- 具体数据结构说明

- **PluginParamsParser**
  - 定义：用于解析插件参数。
  - 注释：解析插件参数为指定的数据类型。
- **Degree**
  - 定义：表示角度的类。
  - 注释：用于处理方向和角度之间的转换。
- **ARPG\_Utils**
  - 定义：实用工具类。
  - 注释：提供各种实用函数，例如查找角色、计算距离等。
- **HitBox**
  - 定义：碰撞盒类。
  - 注释：管理角色的碰撞盒，处理碰撞检测。
- **ComponentRunner**
  - 定义：组件运行管理类。
  - 注释：管理和运行组件的更新。
- **Component**
  - 定义：组件基类。
  - 注释：提供生命周期管理。
- **HitChecker**
  - 定义：碰撞检查类。

- 注释：负责管理和检查碰撞的状态和行为。
- **ErrorManager**
  - 定义：错误管理类。
  - 注释：提供错误处理和错误消息生成。
- **Sprite\_MapWeapon**
  - 定义：武器动画精灵类。
  - 注释：用于地图上显示武器动画的精灵。
  - 取值：

```
var Sprite_MapWeapon = class extends Sprite_Weapon {
    initialize() {
        super.initialize();
        this.inner = new Point();
    }
    initMembers() {
        super.initMembers();
        this.anchor.x = 0.5;
        this.anchor.y = 0.5;
        this.hide();
    }
    update() {
        Sprite.prototype.update.call(this);
    }
};
```

- **SwordAttackProcessor**
  - 定义：剑攻击处理器类。
  - 注释：用于处理剑攻击的动画和逻辑。
  - 取值：

```
var SwordAttackProcessor = class extends
import_Processor.Processor {
    constructor(weaponImgeld) {
```

```

    super();
    this._weaponImgId = weaponImgId;
}
start() {
    super.start();
    const sprite = this.user().getSprite();
    sprite.createMapWeaponSprite(this._weaponImgId);
    this._weaponSprite = sprite._weaponSprite;
}
*process() {
    this._weaponSprite.show();
    switch (this.user().direction()) {
        case 8:
            yield* this.upAttack();
            break;
        case 6:
            yield* this.rightAttack();
            break;
        case 2:
            yield* this.downAttack();
            break;
        case 4:
            yield* this.leftAttack();
            break;
        default:
            throw new Error(`${this.user().direction()} is not found.`);
    }
}
}

```

- 2.3.2 游戏主体

用于图形处理和动画的部分：

- Sprite 类：

- 功能：表示屏幕上的一个图像或动画帧。
- 数据结构：
  - `_blendColor`: 存储 RGBA 颜色值的数组，用于调整图像的混合颜色。
  - `_colorTone`: 存储 RGBA 色调的数组，用于调整图像的色调。
  - `_bitmap`: 关联的 Bitmap 对象，用于渲染图像。
- Bitmap 类：
  - 功能：表示一个可以在屏幕上渲染的位图。
  - 数据结构：
    - `_canvas` 和 `_context`: HTML5 画布和其 2D 渲染上下文，用于位图的绘图操作。
    - `_baseTexture`: 与位图相关联的 PIXI 基础纹理。
    - `_image`: 图像元素，用于加载和显示图像。
    - `_loadingState`: 图像的加载状态，如"none", "loading", "loaded", "error"。
- Tilemap 类：
  - 功能：用于在 2D 游戏中创建和管理瓦片地图。
  - 数据结构：
    - `tileWidth` 和 `tileHeight`:
      - 瓦片的尺寸。
    - `_lowerLayer` 和 `_upperLayer`:
      - 分别用于渲染地图的较低层和较高层的图层。
- 其他部分：
  - `Tilemap.Renderer.prototype`:
    - 地图瓦片渲染器的原型对象，包含初始化方法、销毁方法、着色器创建方法等。
  - `TilingSprite.prototype`:
    - 平铺精灵对象的原型对象，包含初始化方法、销毁方法、更新方法等。
  - `ScreenSprite.prototype`:
    - 屏幕精灵对象的原型对象，包含初始化方法、销毁方法、设置颜色方

法等。

- **Window.prototype:**
  - 窗口对象的原型对象，包含初始化方法、销毁方法、更新方法、设置属性方法等。
- **PluginManager**
  - 是一个静态类，用于管理游戏中的插件。该类包含方法用于加载、设置参数、注册命令以及调用命令等操作。
- 具体数据结构说明：
  - **\_scripts:**
    - 定义：存储已加载的插件脚本的文件名列表。
    - 注释：用于跟踪已加载的插件脚本，避免重复加载。
  - 取值：数组，包含已加载的插件脚本的文件名。
  - **\_errorUrls:**
    - 定义：存储加载插件脚本时出错的 URL 列表。
    - 注释：用于记录加载插件脚本时发生错误的 URL，方便调试和处理错误。
    - 取值：数组，包含发生加载错误的 URL。
  - **\_parameters:**
    - 定义：存储插件参数的对象，键为插件名，值为参数对象。
    - 注释：用于存储各个插件的参数，方便在游戏中获取和使用。
    - 取值：对象，键为插件名（小写），值为参数对象。
  - **\_commands:**
    - 定义：存储注册的命令的对象，键为插件名和命令名的组合，值为对应的函数。
    - 注释：用于存储注册的插件命令及其对应的处理函数，方便在游戏中调用。
    - 取值：对象，键为插件名和命令名的组合，值为对应的函数。
- **2.3.3 玩家管理**
  - 后端：
    - **users** 表：数据库表，用于存储用户信息。

```
SQL
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- `request.json`: 客户端传递的 JSON 数据，用于注册和登录。

```
JSON
{"username": "example", "password": "password", "email": "example@example.com"}
```

- 前端：
  - 登录和注册表单的数据结构。

```
HTML
<input type="text" id="login-username" placeholder="用户名"
required<input type="password" id="login-password"
placeholder="密码" required<input type="text" id="register-
username" placeholder="用户名" required<input
type="password" id="register-password" placeholder="密码"
required<input type="email" id="register-email"
placeholder="邮箱" required
```

- 2.3.4 FLAP 形式语言与自动机解密
  - `master_context`: 从 `AutomataContext` 中获取的上下文对象，存储全局状态和文法对象。
  - `packet_to_misha_the_fasting_juggernaut`: 用于存储和传输到 API 的数据包。
  - `Grammar`, `Rule_Dec`, `T`, `NT`: 用于构建和处理文法的数据结构。
  - `React` 组件: `HeaderMenu` 是一个 `React` 函数组件，用于渲染应用的头部菜单。
  - `useRef`: 创建可变的 `ref` 对象，其 `.current` 属性可被修改，不会引发组件重新渲染。
  - `useState`: 创建状态和对应的设置函数，状态更新会触发组件重新渲染。

- ChatGPT-Agent 插件

- messages:

```
JavaScript
let messages = [
  {"role": "system","content": `World setting:
${worldSetting}. Current plot: ${plot}.`},
  ...dialogHistory.map(entry => ({"role": entry.speaker ===
"Player" ? "user" : "assistant","content": entry.message
})),
  {"role": "user","content": message
}
];
```

- payload:

```
JavaScript
let payload = {model: modelName,messages: messages
};
```

- response:

```
JavaScript
let response = await fetch(url, {method: 'POST',headers:
{'Content-Type': 'application/json','Authorization': 'Bearer
'apiKey
},body: JSON.stringify(payload)
});
let data = await response.json();
```

- 类型: 对象

- 内容:

API 的响应数据。

- data:

```
JavaScript
let data = await response.json();
```

- entry (在 dialogHistory.map 中使用) :

```
JavaScript
```



```
...dialogHistory.map(entry => ({"role": entry.speaker ===  
"Player" ? "user" : "assistant", "content": entry.message  
})))
```

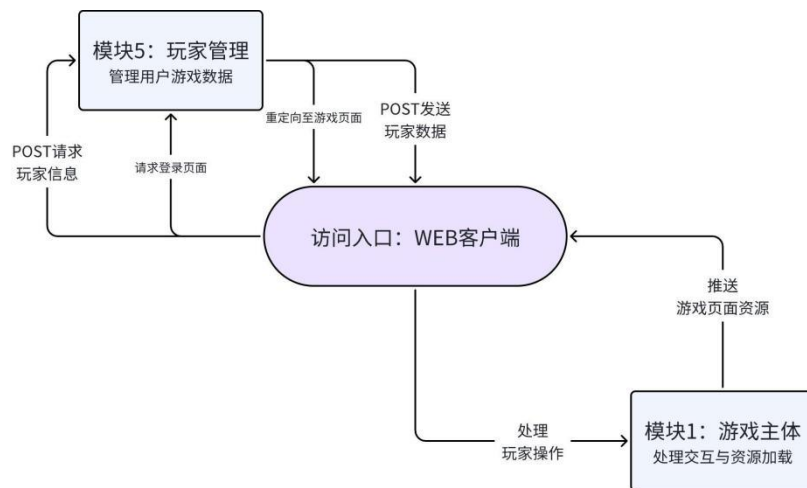
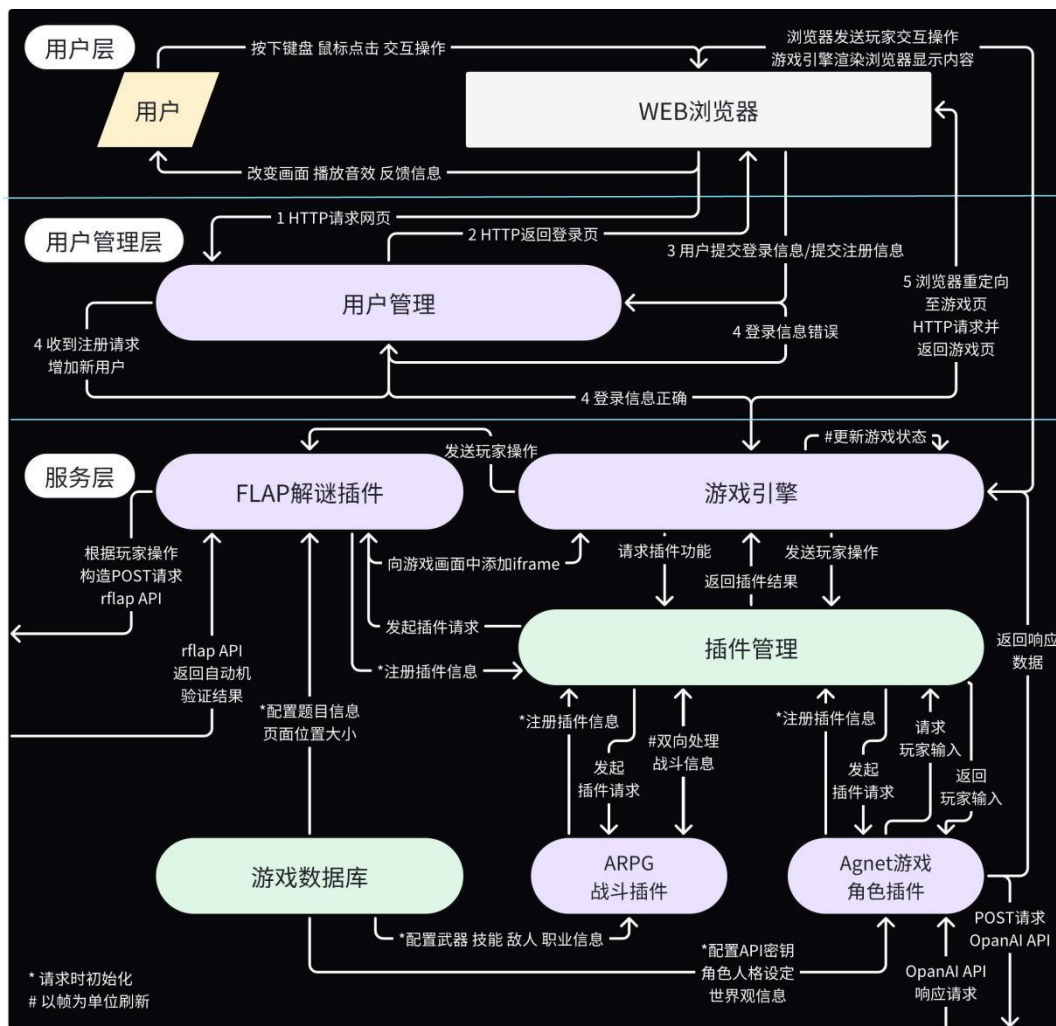
- `args` (在 `PluginManager.registerCommand` 中使用) :

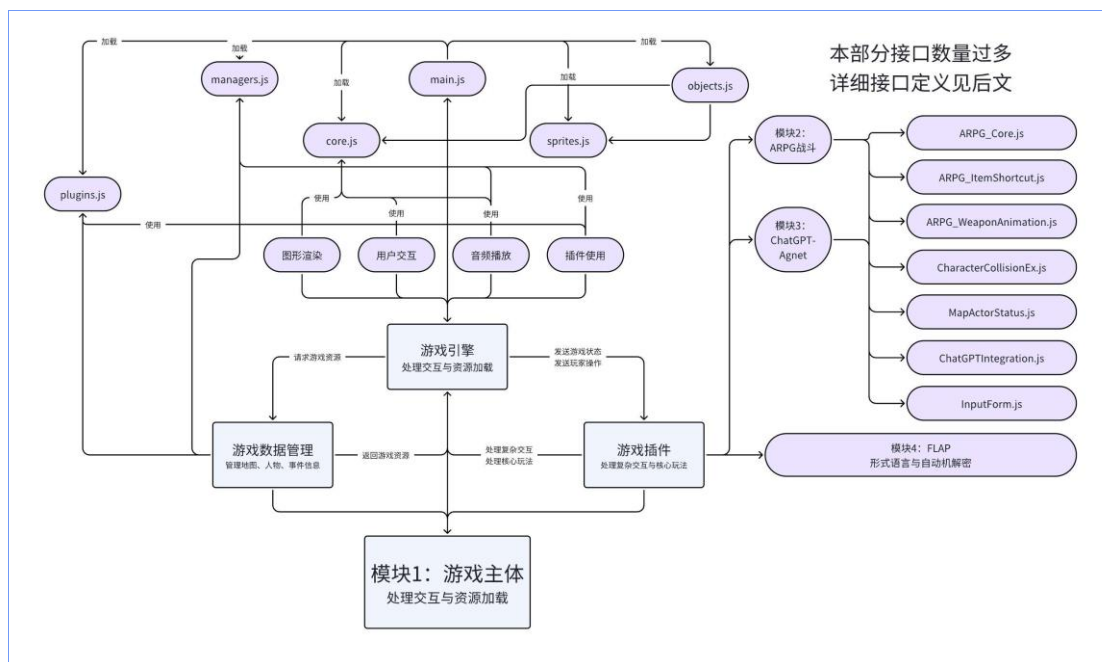
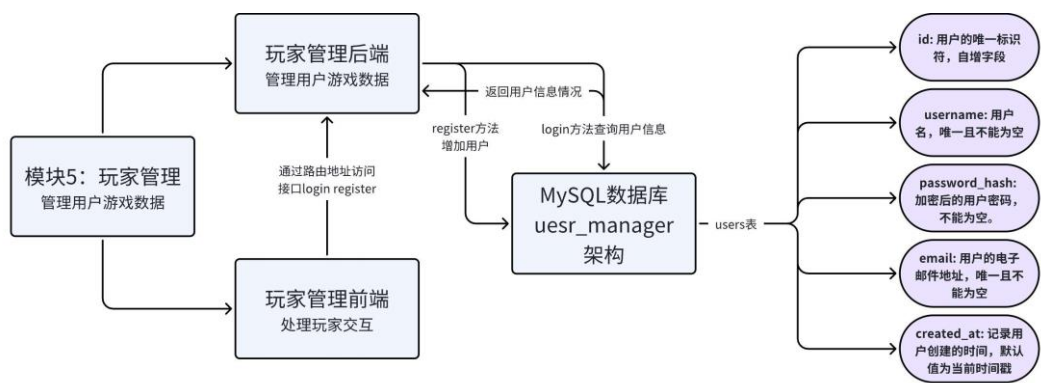
```
JavaScript  
PluginManager.registerCommand('ChatGPTIntegration',  
'SendMessage', args => {const message =  
$gameVariables.value(Number(args.inputId));const currentPlot =  
args.currentPlot;sendMessage(message,  
currentPlot).then(response => {  
    $gameVariables.setValue(Number(args.outputId),  
response);console.log($gameVariables.value(Number(args.outputI  
d)))const info = $gameVariables.value(args.outputId);const  
formattedMessage = info.replace(/(.{29})/g, '$1\n');  
    $gameMessage.add(formattedMessage);  
    });  
});
```

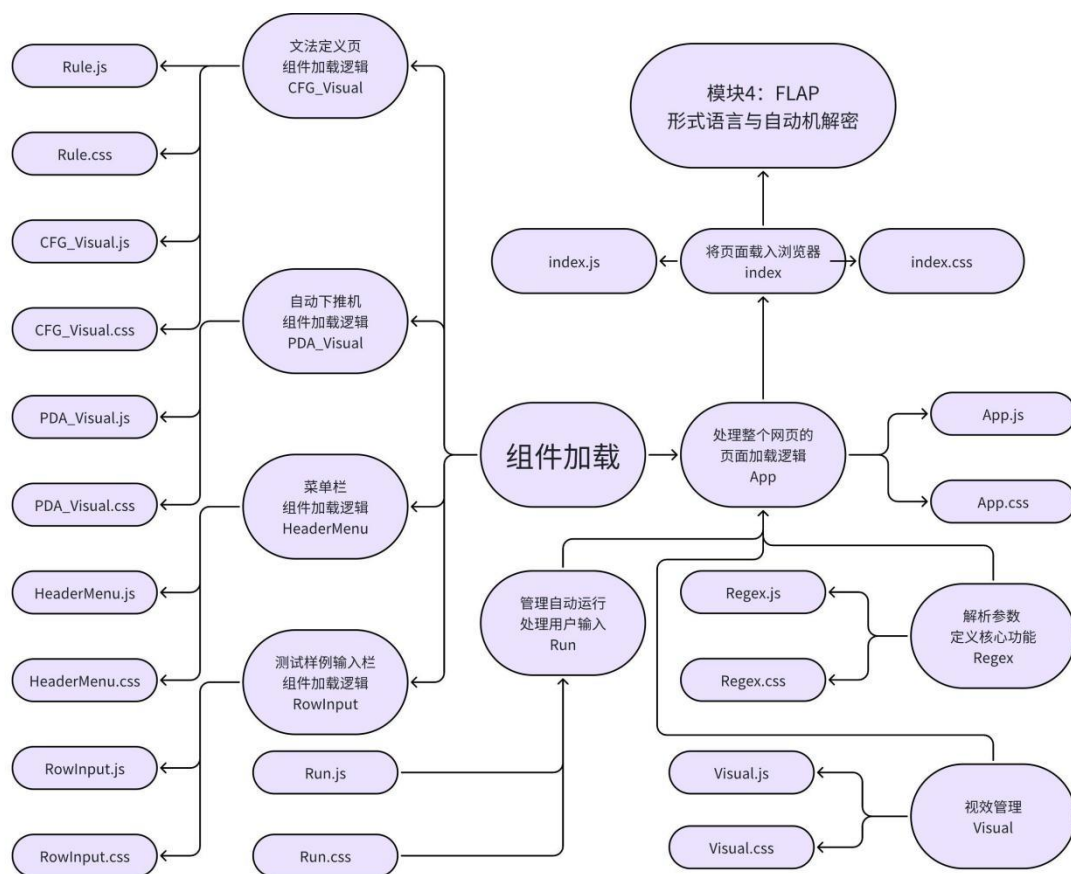
## 3、模块设计

### 3.1、软件结构

按模块划分本项目，可分为游戏引擎与玩家管理两个大部分，细分为游戏主体、ARPG 战斗插件、Agent 插件、FLAP 插件和玩家管理四个部分。具体软件结构与关系如下：







### 3.2、功能设计说明

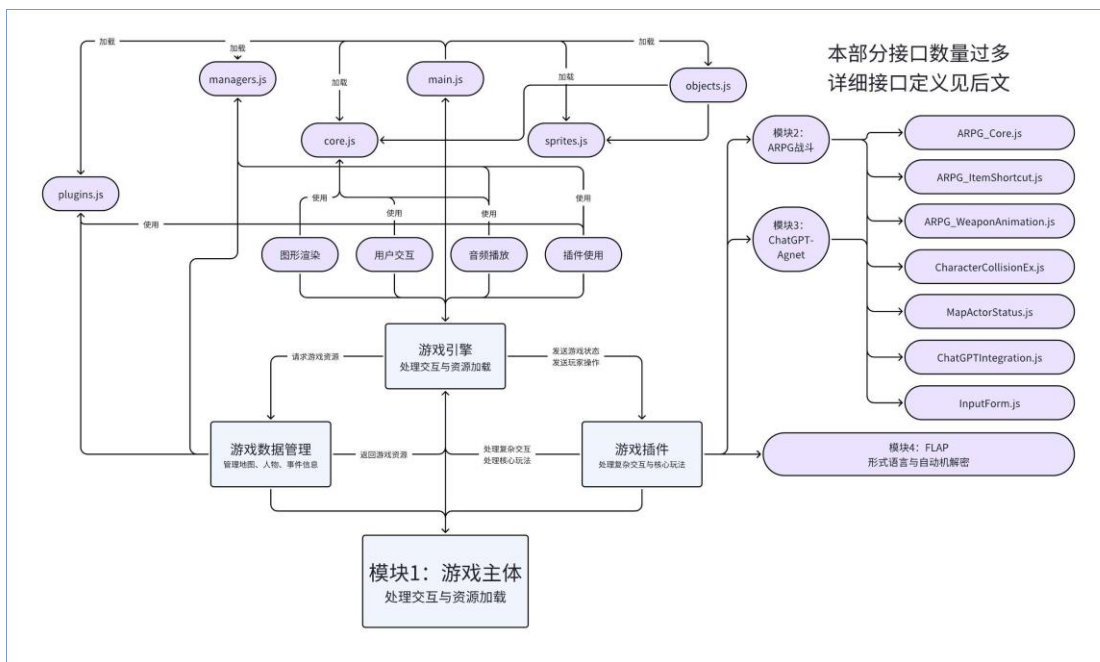
本软件的基本设计思想为模块化开发。不同模块负责不同的功能。模块间低耦合，不同模块通过定义好的接口通信，便于并行开发，分别维护。

并且，插件模块具有高可扩展性，游戏主体可以通过简单的配置文件修改，达成复杂的游戏插件添加，而无需修改游戏主体本身。游戏插件也无需跟随游戏主体变化而变化，具有高度可移植性。

模块间关系呈树状结构，叶节点为 ARPG 战斗插件、Agent 插件、FLAP 插件，分别实现不同的细化功能，并被父节点游戏主体模块统一管理并加载。玩家管理模块作为入口模块，连接至游戏主体模块，并管理玩家信息，可以单独维护管理，减少了后端压力。

### 3.3、模块 1：游戏主体

#### 3.3.1、设计图



### 3.3.2 功能描述 (Function Description)

管理游戏数据和全局信息，负责数据的加载、保存以及错误处理。

### 3.3.3 输入数据 (Input Data)

#### 3.3.3.1 用户输入:

用户通过游戏界面进行的输入，如角色移动指令、战斗选项选择等。

#### 3.3.3.2 有效性检验规则:

输入数据经过格式和逻辑验证，确保不会引发程序错误或者不一致状态。

#### 3.3.3.3 数据来源:

主要从\$gamePlayer、\$gameParty 等对象获取游戏状态数据。

#### 3.3.3.4 获取条件:

根据游戏场景和用户交互状态动态读取。

### 3.3.4 输出数据 (Output Data)

#### 3.3.4.1 产生的数据:

包括角色状态更新、游戏进度存档、战斗结果等。

#### 3.3.4.2 表现形式:

数据通过游戏界面元素动态展示，如角色血量条、经验值等。

### 3.3.5 数据设计 (Data Design)

#### 3.3.5.1 局部数据结构:

名称: \$gameVariables

功能: 存储游戏中的全局变量。

结构: 数组索引存储各变量值, 支持多种数据类型 (整数、字符串等)。

#### 3.3.5.2 数据库表设计:

无实际数据库表, 使用 JSON 文件 (如 Actors.json、Skills.json 等) 模拟数据库功能。

#### 3.3.5.3 数据存储设计:

文件名: global.json

数据格式: JSON

数据项: 包括角色数据、技能、装备、敌人等。

### 3.3.6 算法和流程 (Algorithm and Process)

#### 3.3.6.1 算法描述:

根据玩家输入计算角色行动结果, 如攻击命中与否、伤害值计算等。

#### 3.3.6.2 流程:

读取输入

验证数据有效性

根据游戏逻辑处理数据

输出处理结果到游戏界面

保存游戏状态

### 3.3.7 函数说明 (Function Description)

- loadDatabase

- 功能: 负责加载游戏所需的所有数据文件。
- 格式: loadDatabase()
- 参数: 无
- 全局变量: 使用了多个全局变量, 如 \$dataActors, \$dataClasses 等。
- 局部变量: xhr, url 等。
- 返回值: 无
- 算法说明: 使用 XMLHttpRequest 加载指定的 JSON 文件。
- 使用约束: 必须在游戏初始化时调用。

- Game\_BattlerBase.prototype.paramBase

- 功能：获取角色的基础参数值。
- 格式：paramBase(paramId)
- 参数：paramId (Number) 要获取的基础参数的 ID。
- 全局变量：无
- 局部变量：无
- 返回值：(Number) 基础参数值。
- 算法说明：直接返回一个固定的值，通常为 0，因为该函数为基类方法，具体的参数值由继承类实现。
- 使用约束：在继承类中被覆盖以提供具体的参数值。
- Game\_BattlerBase.prototype.paramPlus
  - 功能：获取角色参数的加成值。
  - 格式：paramPlus(paramId)
  - 参数：paramId (Number) 要获取的参数加成的 ID。
  - 全局变量：无
  - 局部变量：无
  - 返回值：(Number) 参数的加成值。
  - 算法说明：返回角色的参数加成数组中对应 ID 的元素。
  - 使用约束：通常与 paramBase 一起使用，以计算角色的最终参数值。
- Game\_BattlerBase.prototype.param
  - 功能：计算并返回角色的最终参数值。
  - 格式：param(paramId)
  - 参数：paramId (Number) 要计算的参数的 ID。
  - 全局变量：无
  - 局部变量：无
  - 返回值：(Number) 角色的最终参数值。
  - 算法说明：将基础参数值与加成值相加，并乘以角色特性中对该参数的倍率，最后乘以 buff 的影响。
  - 使用约束：用于获取角色的最终参数，如攻击力、防御力等。
- Game\_BattlerBase.prototype.xparam
  - 功能：获取角色的扩展参数值。

- 格式: xparam(xparamId)
- 参数: xparamId (Number) 要获取的扩展参数的 ID。
- 全局变量: 无
- 局部变量: 无
- 返回值: (Number) 扩展参数值。
- 算法说明: 通过角色特性中对该扩展参数的总和来计算。
- 使用约束: 用于获取角色的扩展参数, 如命中率、回避率等。
- Game\_BattlerBase.prototype.sparam
  - 功能: 获取角色的特定参数值。
  - 格式: sparam(sparamId)
  - 参数: sparamId (Number) 要获取的特定参数的 ID。
  - 全局变量: 无
  - 局部变量: 无
  - 返回值: (Number) 特定参数值。
  - 算法说明: 计算角色特性中对该特定参数的累乘结果。
  - 使用约束: 用于获取角色的特定参数, 如战斗后经验增加率等。
- Game\_BattlerBase.prototype.elementRate
  - 功能: 获取角色对某个元素的抗性倍率。
  - 格式: elementRate(elementId)
  - 参数: elementId (Number) 要获取抗性的元素的 ID。
  - 全局变量: 无
  - 局部变量: 无
  - 返回值: (Number) 元素抗性倍率。
  - 算法说明: 计算角色特性中对该元素抗性的累乘结果。
  - 使用约束: 用于确定角色受到特定元素伤害时的伤害倍率。
- Game\_BattlerBase.prototype.stateRate
  - 功能: 获取角色对某个状态的抗性倍率。
  - 格式: stateRate(stateId)
  - 参数: stateId (Number) 要获取抗性的状态的 ID。



- 全局变量：无
- 局部变量：无
- 返回值：（Number）状态抗性倍率。
- 算法说明：计算角色特性中对该状态抗性的累乘结果。
- 使用约束：用于确定角色受到特定状态影响的概率。
- `Game_BattlerBase.prototype.buffRate`
  - 功能：获取角色的增益效果倍率。
  - 格式：`buffRate(paramId)`
  - 参数：`paramId`（Number）要获取增益效果倍率的参数的 ID。
  - 全局变量：无
  - 局部变量：无
  - 返回值：（Number）增益效果倍率。
  - 算法说明：返回角色 buff 数组中对应 ID 的元素与 1 的和的四分之一。
  - 使用约束：用于计算角色的增益 buff 效果

## Scene\_Base

- 函数名称: `initialize`
  - 所在文件: `Scene_Base`
  - 功能: 构造函数，初始化场景基础属性。
  - 格式: `function Scene_Base() { ... }`
  - 参数: 无
  - 全局变量: 无
  - 局部变量: `this._started`, `this._active`, 等
  - 返回值: 无
  - 算法说明: 初始化场景状态标志和渐变效果相关属性。
  - 使用约束: 作为场景类的基础，被其他所有场景类继承。
- 函数名称: `start`
  - 所在文件: `Scene_Battle`
  - 功能: 开始战斗场景，播放战斗背景音乐，启动战斗。
  - 格式: `Scene_Battle.prototype.start = function() { ... }`

- 参数: 无
- 全局变量: BattleManager
- 局部变量: 无
- 返回值: 无
- 算法说明: 设置战斗音乐, 开始战斗逻辑, 淡入效果。
- 使用约束: 需要与战斗管理器 (BattleManager) 协同工作。

## Scene\_Shop

- 函数名称: prepare
  - 所在文件: Scene\_Shop
  - 功能: 准备商店场景所需的商品数据。
  - 格式: Scene\_Shop.prototype.prepare = function(goods, purchaseOnly) { ... }
  - 参数: goods (商品数组), purchaseOnly (是否仅购买)
  - 全局变量: 无
  - 局部变量: this.\_goods, this.\_purchaseOnly
  - 返回值: 无
  - 算法说明: 存储商品数据和购买标志。
  - 使用约束: 在创建商店场景前必须调用, 以确保数据准备就绪。

## Scene\_Name

- 函数名称: prepare
  - 所在文件: Scene\_Name
  - 功能: 准备名称输入场景所需的角色 ID 和最大字符长度。
  - 格式: Scene\_Name.prototype.prepare = function(actorId, maxLength) { ... }
  - 参数: actorId (角色 ID), maxLength (最大字符长度)
  - 全局变量: \$gameActors
  - 局部变量: this.\_actorId, this.\_maxLength
  - 返回值: 无
  - 算法说明: 设置角色 ID 和名称的最大长度, 用于名称输入。
  - 使用约束: 在创建名称输入场景前必须调用。

## Scene\_Debug

- 函数名称: create
  - 所在文件: Scene\_Debug
  - 功能: 创建调试场景的窗口和控件。
  - 格式: Scene\_Debug.prototype.create = function() { ... }
  - 参数: 无
  - 全局变量: 无
  - 局部变量: 无
  - 返回值: 无
  - 算法说明: 创建范围选择窗口、编辑窗口和调试帮助窗口。
  - 使用约束: 作为调试场景的一部分, 用于开发者测试和调试游戏。

## Scene\_Gameover

- 函数名称: start
  - 所在文件: Scene\_Gameover
  - 功能: 开始游戏结束屏幕, 播放游戏结束音乐, 淡入效果。
  - 格式: Scene\_Gameover.prototype.start = function() { ... }
  - 参数: 无
  - 全局变量: \$dataSystem
  - 局部变量: 无
  - 返回值: 无
  - 算法说明: 加载游戏结束背景, 调整背景显示, 淡入效果。
  - 使用约束: 游戏结束时显示, 提供返回标题屏幕的选项。

### 3.3.8 全局数据结构与该模块的关系 (Relation to Global Data Structures)

访问的全局数据结构:

- **\$gameParty**: 存储游戏队伍的相关信息, 如角色列表、金钱、物品等。
- **\$gameMap**: 存储当前游戏地图的信息, 如地图 ID、事件、车辆等。
- **\$gameSystem**: 存储游戏系统设置, 如音乐、音效、窗口色调等。
- **\$gameTemp**: 存储临时游戏数据, 如动画、气球、战斗结果等。

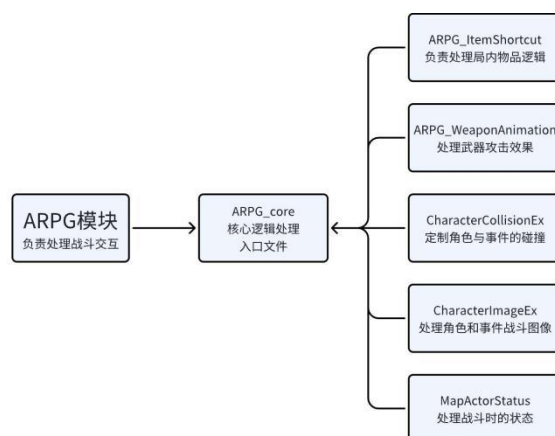
关系说明: DataManager 负责初始化、加载和处理这些全局数据结构, 确保游戏

数据的一致性和可用性。

### 3.4、模块 2：ARPG 战斗插件

详细描述各功能模块的功能、数据结构、具体算法和流程等。

#### 3.4.1、设计图



#### 3.4.2、功能描述

该 ARPG 功能模块主要用于管理游戏中的动作角色扮演（ARPG）元素，通过一系列插件命令，实现角色行为、事件动态生成、技能使用、碰撞检测等功能。具体功能包括：

- 模式切换：启动或结束 ARPG 模式。
- 动态事件生成：在特定位置生成新的动态事件。
- 角色位置获取：获取角色的当前位置或中心位置。
- 角度与距离计算：计算两个角色之间的角度和距离。
- 屏幕内检测：检查角色是否在屏幕内。
- 移动检测：检测角色是否移动。
- 敌人设置：设置事件为敌人，并配置其血量条等属性。
- 血量条显示切换：显示或隐藏角色的血量条。
- 场景对象设置：设置事件为场景对象。
- 技能与物品使用：角色使用技能或物品。
- 技能效果应用：应用技能效果，或测试技能效果是否生效。
- 技能对象生成：生成技能对象，并设置其位置。
- 命中盒设置与检测：设置角色的命中盒，并进行命中检测。

- 角色属性获取与设置：获取或设置角色的特定属性和参数。
- 击退与动作等待：处理角色击退效果和动作等待。

### 3.4.3、输入数据

- 输入数据主要来自插件命令参数、游戏变量和开关，具体包括：
  - 命令参数：通过插件命令调用时传入的参数，如角色规格、位置、角度等。
  - 游戏变量：用于动态获取或设置一些参数，如位置变量、技能 ID 变量等。
  - 游戏开关：用于存储布尔结果，如命中检测结果、角色移动检测结果等。
- 输入数据有效性检验规则
  - 参数类型验证：确保每个参数的类型（如 number, string, boolean）正确。
  - 变量与开关验证：确保游戏变量和开关在有效范围内，并且已初始化。

### 3.4.4、输出数据

输出数据主要是通过游戏变量和开关进行存储和表现，具体包括：

- 位置数据：存储角色的位置坐标。
- 角度和距离数据：存储角色之间的角度和距离。
- 命中检测结果：布尔值，表示是否命中目标。
- 角色属性数据：存储角色的血量、魔法值、技能取消率等。

### 3.4.5、数据设计

- 局部数据结构
  - 角色规格（CharacterSpecification）：描述角色的识别方式。
    - CharacterSpecification：角色 ID 或名称，用于唯一标识角色。
  - 位置结构（PositionStructure）：描述角色或事件的位置。
    - X：横坐标。
    - Y：纵坐标。
    - XByVariable：用游戏变量指定横坐标。
    - YByVariable：用游戏变量指定纵坐标。
- 数据库表与数据存储设计
  - 游戏变量表：存储动态获取或设置的游戏数据。

- 游戏开关表：存储布尔值结果，表示不同逻辑状态。

#### 3.4.6、算法和流程

- ARPG 模式切换：
  - 输入：ARPGMode 参数。
  - 流程：根据 ARPGMode 参数启动或结束 ARPG 模式。
- 动态事件生成：
  - 输入：SrcMapId, SrcEventIdOrName, X, Y, XByVariable, YByVariable, MadeDynamicEventId。
  - 流程：
    - i. 解析位置参数。
    - ii. 生成动态事件。
    - iii. 如果 MadeDynamicEventId 有效，设置游戏变量为生成的事件 ID。
- 技能使用：
  - 输入：SkillId, SkillIdByVariable, SkillByName。
  - 流程：
    - i. 根据输入参数确定技能 ID 或名称。
    - ii. 调用角色的 useSkill 方法使用技能。

#### 3.4.7、函数说明

- 函数名称：ChangeARPGMode
  - 所在文件：ARPG\_CorePlugin.js
  - 功能：启动或结束 ARPG 模式。
  - 参数：
    - args：包含 ARPGMode 参数的对象。
  - 返回值：无。
  - 算法说明：解析 ARPGMode 参数，根据其布尔值调用 \$gameMap.startARPGMode 或 \$gameMap.endARPGMode 方法。
- 函数名称：MakeDynamicEvent
  - 所在文件：ARPG\_CorePlugin.js

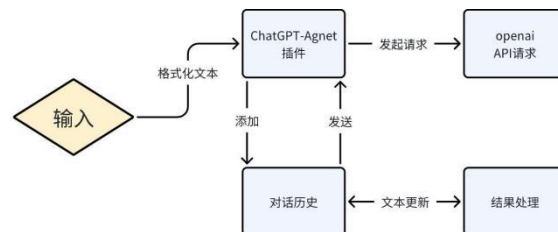
- 功能：在指定位置生成动态事件。
- 参数：
  - `args`：包含位置和事件 ID 等参数的对象。
- 返回值：无。
- 算法说明：解析位置参数，调用 `$gameMap.makeDynamicEvent` 生成事件，并设置游戏变量。

#### 3.4.8 全局数据结构与该模块的关系

- 游戏变量 (`$gameVariables`)：用于获取和设置动态参数。
- 游戏开关 (`$gameSwitches`)：用于存储布尔结果。
- 地图对象 (`$gameMap`)：用于生成动态事件和控制 ARPG 模式。
- 临时数据 (`$gameTemp`)：用于存储临时选择结果和全局参数。

### 3.5、模块 3：ChatGPT-Agent 插件

#### 3.4.1、设计图



#### 3.4.2、功能描述

该模块是一个集成了 OpenAI 的 ChatGPT API 的 RPG Maker 插件。它的主要功能是将玩家输入的信息发送到 ChatGPT，并将生成的响应返回给游戏以实现动态对话。

#### 3.4.3、输入数据

##### 3.4.3.1 玩家输入的数据：

- 输入方式：通过游戏界面输入的文本信息。
- 有效性检验规则：输入数据必须是字符串，长度限制在一定范围内（通常不超过 1000 字符），防止过长的输入影响性能。

##### 3.4.3.2 配置参数：

- `worldSetting`：游戏的世界设定，用于向 ChatGPT 提供上下文信息。

- `apiKey`: OpenAI 的 API 密钥, 用于认证和授权访问 ChatGPT 服务。

#### 3.4.4、输出数据

- 数据形式: 字符串文本。
- 数据表现: ChatGPT 生成的响应将以对话形式显示在游戏界面中。
- 处理: 将响应文本按指定字符数进行换行, 以适应游戏界面的显示格式。

#### 3.4.5、数据设计

##### 3.4.5.1 局部数据结构:

- `parameters`: 插件参数, 包括 `worldSetting` 和 `apiKey`。
- `dialogHistory`: 对话历史数组, 存储玩家和 ChatGPT 之间的对话记录。
- `plot`: 当前剧情文本, 累积玩家的所有输入, 以提供给 ChatGPT 上下文。
- 定义:

```
const parameters = PluginManager.parameters('ChatGPTIntegration');
const worldSetting = parameters['worldSetting'];
const apiKey = parameters['apiKey'];
const dialogHistory = [];
let plot = "";
```

#### 3.4.6、算法和流程

##### 3.4.6.1 发送消息的流程:

- 停止游戏循环: `Graphics.stopGameLoop();`
- 更新剧情文本: `plot = plot + currentPlot`
- 准备 API 请求的消息列表, 包括系统信息、对话历史和玩家当前输入。
- 构建 API 请求的载荷, 包括模型名称和消息列表。
- 发送 API 请求到 OpenAI 的 ChatGPT 服务, 等待响应。
- 解析响应数据, 更新对话历史并重新启动游戏循环:  
`Graphics.startGameLoop();`
- 返回 ChatGPT 生成的响应文本。
- 实现方式:



```
async function sendMessage(message, currentPlot) {
  Graphics.stopGameLoop();
  plot = plot + currentPlot;
  let url = "https://api.openai.com/v1/chat/completions";
  let messages = [
    {
      "role": "system",
      "content": "World setting: ${worldSetting}. Current plot:
${plot}."
    },
    ...dialogHistory.map(entry => ({
      "role": entry.speaker === "Player" ? "user" : "assistant",
      "content": entry.message
    })),
    {
      "role": "user",
      "content": message
    }
  ];
  let payload = {
    model: modelName,
    messages: messages
  };

  let response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer ' + apiKey
    },
    body: JSON.stringify(payload)
  });
```

```

let data = await response.json();
dialogHistory.push({speaker: "Player", message: message});
if (data.choices && data.choices.length > 0) {
    let botResponse = data.choices[0].message.content.trim();
    dialogHistory.push({speaker: "ChatGPT", message: botResponse});
    Graphics.startGameLoop();
    return botResponse;
}
Graphics.startGameLoop();
return "No response from ChatGPT.";
}

```

### 3.4.7、函数说明

#### 3.4.7.1 sendMessage:

- 文件: `ChatGPTIntegration.js`
- 功能: 发送玩家输入给 ChatGPT, 并返回生成的响应。
- 格式: `async function sendMessage(message, currentPlot)`
- 参数:
  - `message`: 玩家的输入信息。
  - `currentPlot`: 当前的剧情文本。
- 局部变量:
  - `url`: ChatGPT API 的请求地址。
  - `messages`: 构建的消息列表。
  - `payload`: API 请求的载荷。
  - `response`: API 响应对象。
  - `data`: 解析后的响应数据。
- 返回值: ChatGPT 生成的响应文本。
- 算法说明: 见 3.4.6 节。
- 使用约束: 需要正确配置 API 密钥和模型名称, 确保网络连接正常。

### 3.4.8 全局数据结构与该模块的关系

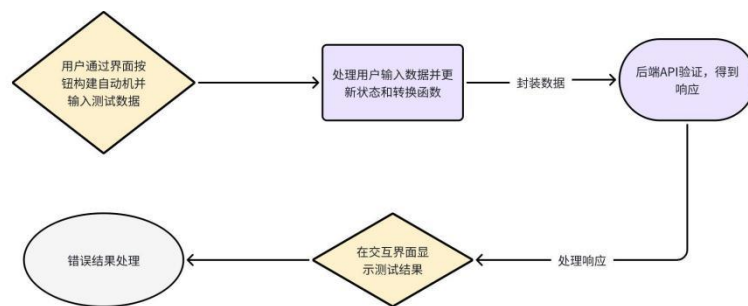
插件中访问的全局数据结构：

- `PluginManager.parameters('ChatGPTIntegration')`：获取插件参数。
- `$gameVariables`：用于存储和传递输入和输出数据。
- `Graphics`：控制游戏循环的启动和停止。

### 3.6、模块 4：FLAP 插件

详细描述各功能模块的功能、数据结构、具体算法和流程等。

#### 3.6.1、设计图



#### 3.6.2、功能描述

FLAP 是一个自动机可视化模块，它主要负责将用户输入的自动机状态和转换关系可视化，提供友好的界面，帮助用户直观理解自动机的运行和状态转移。它的功能包括：

- 绘制状态图：根据用户输入的状态和转换关系，动态绘制自动机状态图。
- 编辑状态和转换：允许用户通过点击和拖拽操作添加、删除或修改状态和转换关系。
- 设置初始和终止状态：用户可以通过点击操作设置某个状态为初始状态或接受状态。

#### 3.6.3、输入数据

##### 3.6.3.1 用户输入数据：

1. 状态（节点）：用户通过点击画布添加状态节点，输入状态名称。
2. 转换（边）：用户通过点击和拖拽操作添加转换边，输入转换条件（字符或字符序列）。
3. 设置初始状态和终止状态：用户通过点击状态节点设置初始状态或接受状态。

### 3.6.3.2 有效性检验规则：

1. 状态名称应唯一且非空。
2. 转换条件应为有效的字符或字符序列，且不能包含空白字符。
3. 初始状态应唯一且必须设置。
4. 接受状态可以有多个，但必须是有效状态节点。

### 3.6.3.3 数据获取：

1. 从用户输入的状态节点和转换边数据构建自动机。

### 3.6.4、输出数据

#### 3.6.4.1 产生的数据：

1. 状态图：显示状态节点和转换边的图形表示。
2. 状态和转换信息：以文本形式显示当前自动机的状态节点和转换关系。

#### 3.6.4.1 数据表现形式：

1. 图形界面：在画布上以图形化方式展示自动机的状态节点和转换边。
2. 文本输出：在侧边栏或弹窗中以文本形式显示当前自动机的状态和转换信息。

### 3.6.5、数据设计

详见 2.3.4

### 3.6.6、算法和流程

FLAP 插件主要用于用户输入测试和考试模式的处理，以及自动机的导入、导出和处理。该模块包括用户界面交互、自动机状态和转换的处理、数据提交到后端 API 以及错误处理和反馈。

#### 3.6.6.1 流程概述

1. 初始化
  - 加载必要的库和资源，包括 `vis-network`、`react-bootstrap` 等。
  - 初始化状态节点数据集和转换边数据集。
  - 设置全局变量和初始错误对象。
2. 用户界面交互
  - 用户通过界面按钮添加输入行。

- 用户输入测试或考试数据。
- 用户点击按钮提交数据进行测试或考试。

### 3. 数据处理

- 处理用户输入数据并更新状态和转换函数。
- 将数据封装成请求对象，并发送到后端 API 进行验证。
- 处理 API 响应，显示测试或考试结果。

### 4. 错误处理

- 检查用户输入和状态转换函数中的错误。
- 根据错误对象显示相应的错误信息。

## 3.6.7、函数说明

### 3.6.7.1 App

1. 功能：一个 React 组件，负责管理和显示不同类型的自动机的可视化界面。组件利用了多种 React 特性，如状态管理、事件处理、条件渲染和上下文管理。
2. 参数和返回值
  - 参数：无
  - 返回值：返回 JSX 元素，用于构建整个应用的 UI 界面。
3. 全局变量和局部变量
  - 全局变量：
    - `master_context`：包含自动机的全局状态信息，如图形对象、文法规则数组和标志位。
    - `CURRENT_MACHINE`：枚举，定义了不同类型自动机的标识。
  - 局部变量：
    - `sidebar_display`：布尔值状态，控制侧边栏组件的显示与隐藏。
    - `machine_displayed`：枚举值状态，表示当前选中显示的自动机类型。
    - `lock_run_button`：布尔值，用于防止“运行”按钮导致的无限渲染循环。
4. 主要函数和方法
  - `scheduleMeasurement`：函数，用于启动应用性能的测量。

- `uuidv4`: 函数，生成唯一的会话 ID。
- `useState`: React 钩子，用于状态管理。
- `useCallback`: React 钩子，用于缓存函数，避免不必要的重新渲染。
- `useEffect`: React 钩子，用于处理组件的副作用，如事件监听和数据订阅。
- `click_run_handler`: 事件处理函数，根据用户点击行为更新侧边栏的显示状态。
- `render_machine_display`: 函数，根据当前选择的自动机类型动态渲染相应的视图组件。

## 5. 算法说明

- 初始化：在组件加载后，立即进行性能测量并设置会话 ID 和日期。
- 状态管理：使用 React 的 `useState` 钩子来管理侧边栏的显示状态和当前显示的自动机类型。
- 事件处理：为整个窗口添加点击事件监听，根据点击的目标更新状态或触发侧边栏的显示/隐藏。
- 条件渲染：基于状态变量动态渲染侧边栏和不同类型自动机的视图组件。
- 资源清理：使用 `useEffect` 的返回函数来清理事件监听器，避免内存泄露。

## 6. 使用约束

- 确保所有用到的资源和依赖（如 CSS 文件、外部组件和上下文）被正确导入和配置。
- 确保环境支持最新的 JavaScript 和 React 特性。
- `master_context` 和 `CURRENT_MACHINE` 需要在组件外部正确定义和初始化。

### 3.6.7.2 postToRustApi

1. 功能：向后端 API 发送 POST 请求。
2. 格式：异步函数。
3. 参数：无。
4. 局部变量：`url`（API 的 URL）、`postingObject`（POST 请求的配置对象）。
5. 返回值：API 响应的 JSON 数据。
6. 算法说明：构建 POST 请求并发送，等待响应并返回。

#### 3.6.7.3 definition\_plus\_handler

1. 功能：添加新的语法规则行到界面和状态数组中。
2. 格式：普通函数。
3. 参数：`button_press`（触发此函数的按钮事件对象）。
4. 局部变量：`array_to_mount`（当前的定义数组）、`grammar_table_line`（新的文法行对象）。
5. 返回值：无。
6. 算法说明：更新定义数组和全局上下文的文法对象。
7. 约束条件：确保所有用到的资源和依赖（如 CSS 文件、外部组件和上下文）被正确导入和配置。

#### 3.6.7.4 UIN\_submit

1. 功能：提交用户输入，校验并处理数据导出。
2. 格式：普通函数。
3. 参数：`event`（触发此函数的事件对象）。
4. 局部变量：`input_val`（输入值）、`append`（随机数，用于文件名）、`temp_pack`（处理后的数据包）。
5. 返回值：无。
6. 算法说明：校验输入格式，正确时处理并导出数据，否则显示警告。
7. 约束条件：需要处理的字符串和数组操作要注意性能影响，特别是在大数据集上。

#### 3.6.7.5 HeaderMenu

1. 功能：渲染头部导航菜单，并处理用户交互。
2. 参数：`props` — React 组件属性。
3. 返回值：返回渲染的 React 元素。
4. 算法说明：组件通过 `useEffect` 监听 DOM 修改和全局状态变化，并通过事件处理函数响应用户操作。

#### 3.6.7.6 nav\_menu\_dropdown\_click

1. 功能：点击下拉菜单项时更新当前选择的自动机类型。
2. 格式：函数。

3. 参数:
  - a. `e`: 事件对象。
  - b. `machine`: 对应自动机类型的 `useRef` 对象。
4. 局部变量: `name` — 根据点击的菜单项解析出的自动机名称。
5. 算法说明: 根据点击的菜单项更新全局状态 `master_context` 和本地状态 `machine_select`。

#### 3.6.7.7 trimSvgWhitespace

1. 功能: 调整 SVG 图形以确保所有子元素可见。
2. 格式: 函数。
3. 参数: 无。
4. 局部变量: `svgs` — 获取的 SVG DOM 节点数组, `box` — SVG 的视觉边界框, `viewBox` — 根据边界框设置的视图框。
5. 返回值: 无。
6. 算法说明: 遍历每个 SVG 元素, 获取其边界框并设置 `viewBox` 属性以确保内容完整显示。

#### 3.6.7.8 ReactDOM.render

1. 功能: 将 React 组件渲染到 DOM 中。
2. 格式: `ReactDOM.render(component, container[, callback])`
3. 参数:
  - a. `component`: 要渲染的 React 元素。
  - b. `container`: DOM 元素, React 元素将被挂载在这里。
  - c. `callback`: 可选的回调函数, 在组件渲染或更新完成后被调用。
4. 全局变量: 无。
5. 局部变量: 无。
6. 返回值: 无 (对于大多数情况, 这个调用不关心返回值, 主要是触发 React 组件的挂载流程)。
7. 算法说明: 此函数负责初始化 React 组件的渲染流程, 包括组件生命周期的启动、虚拟 DOM 的创建和实际 DOM 的更新。
8. 使用约束: 必须在浏览器环境中使用, 因为它依赖于 DOM。



### 3.6.7.9 useEffect Hook

1. 功能: 初始化网络图, 设置事件监听器, 并在组件卸载时进行清理。
2. 算法说明: 使用 `vis.Network` 创建图形界面, 注册多种事件处理函数, 如点击、拖拽结束等, 用于更新图形状态或响应用户操作。
3. 使用约束: 需要确保所有 DOM 元素在使用前已正确挂载, 且相关资源 (如图片) 已加载。

### 3.6.7.10 deselectAllModes

1. 功能: 重置所有操作模式。
2. 格式: `function deselectAllModes()`
3. 全局变量: `in_initial_mode`, `in_accepting_mode`, `delete_lock`, `img_status`
4. 算法说明: 将所有模式标志变量重置为 `false`, 并更新图标状态。

### 3.6.7.11 nodesOfEdgeId

1. 功能: 根据边 ID 获取起始节点和终止节点的标签及边的文本。
2. 格式: `function nodesOfEdgeId(edgeID)`
3. 参数: `edgeID` — 边的 ID。
4. 全局变量: `graph`
5. 局部变量: `fromLabel`, `toLabel`, `edgeText`, `nodeFromId`, `nodeToId`
6. 返回值: 包含边文本、起始节点标签和终止节点标签的对象。
7. 算法说明: 遍历 `graph.edges` 和 `graph.nodes`, 找到匹配的边和节点, 返回相关信息。

### 3.6.7.12 ChangeEdgeText

1. 功能: 更新边的文本信息。
2. 格式: `function ChangeEdgeText(userInput, edgeID)`
3. 参数: `userInput` — 用户输入的文本, `edgeID` — 边的 ID。
4. 全局变量: `graph`, `edgesDS`, `modalEntry`
5. 局部变量: `warn`, `finalStr`, `warnString`

6. 返回值: 无。
7. 算法说明: 验证用户输入是否合法, 更新边的文本信息, 如果输入不合法则显示警告。

#### 3.6.7.13 findEdgeByNodes

1. 功能: 根据起始节点和终止节点 ID 查找边 ID。
2. 参数: `from` — 起始节点 ID, `to` — 终止节点 ID。
3. 全局变量: `graph`
4. 局部变量: `return_id`
5. 返回值: 返回找到的边 ID。
6. 算法说明: 遍历 `graph.edges`, 找到匹配的边并返回其 ID。

#### 3.6.7.14 toEditEdgeMode

1. 功能: 切换到编辑边模式。
2. 参数: `props` — 属性对象。
3. 格式: `function toEditEdgeMode(props)`
4. 全局变量: `network`, `img_status`
5. 算法说明: 重置所有模式, 启用边编辑模式, 并更新图标状态。

#### 3.6.7.15 toAddNodeMode

1. 功能: 切换到添加节点模式。
2. 格式: `function toAddNodeMode(props)`
3. 参数: `props` — 属性对象。
4. 全局变量: `network`, `img_status`
5. 算法说明: 重置所有模式, 启用节点添加模式, 并更新图标状态。

#### 3.6.7.16 setInitial

1. 功能: 设置起始状态模式。
2. 格式: `function setInitial(props)`
3. 参数: `props` — 属性对象。

4. 全局变量: `in_initial_mode`, `img_status`
5. 算法说明: 重置所有模式, 启用起始状态模式, 并更新图标状态。

#### 3.6.7.17 setAccepting

1. 功能: 设置接受状态模式。
2. 格式: `function setAccepting(props)`
3. 参数: `props` — 属性对象。
4. 全局变量: `in_accepting_mode`, `img_status`
5. 算法说明: 重置所有模式, 启用接受状态模式, 并更新图标状态。

#### 3.6.7.18 newNodeLabel

1. 功能: 生成新节点的标签。
2. 格式: `function newNodeLabel()`
3. 全局变量: `nodesDS`
4. 局部变量: `returnLabel`, `nominalAppend`, `foundEmptyIndex`, `nodesPresent`
5. 返回值: 新节点的标签。
6. 算法说明: 生成一个唯一的节点标签, 确保标签不与现有节点重复。

#### 3.6.7.19 populateNode

1. 功能: 添加新节点。
2. 格式: `function populateNode(props)`
3. 参数: `props` — 属性对象。
4. 全局变量: `node_id_global`, `nodesDS`, `network`
5. 算法说明: 生成新节点 ID 和标签, 并将节点添加到图中。

#### 3.6.7.20 deleteNodeOrEdge

1. 功能: 删除选中的节点或边。
2. 格式: `function deleteNodeOrEdge(props)`
3. 参数: `props` — 属性对象。
4. 全局变量: `network`

5. 算法说明: 删除选中的节点或边。

#### 3.6.7.21 promptFile

1. 功能: 提示用户选择文件并读取文件内容。
2. 格式: `function promptFile(contentType, multiple)`
3. 参数: `contentType` — 文件类型, `multiple` — 是否允许多选。
4. 返回值: 返回一个 `Promise`, 在文件读取完成时解析。
5. 算法说明: 创建文件输入元素, 监听文件选择事件, 并读取文件内容。

#### 3.6.7.22 animateIntoNeutral

1. 功能: 将按钮状态动画设置为中性。
2. 格式: `function animateIntoNeutral(status_ref, test_button_ref)`
3. 参数: `status_ref` — 状态引用, `test_button_ref` — 按钮引用。
4. 算法说明: 移除按钮的旋转动画类。

#### 3.6.7.23 checkForProperDeterminism

1. 功能: 检查确定性。
2. 格式: `function checkForProperDeterminism(listOfStringsCallback, single_entry)`
3. 参数: `listOfStringsCallback` — 回调列表, `single_entry` — 是否为单个条目。
4. 返回值: 布尔值, 表示是否符合确定性。
5. 算法说明: 遍历回调列表, 检查是否符合确定性。

#### 3.6.7.24 onClickPingToApi

1. 功能: 处理 API 请求的点击事件。
2. 格式: `async function onClickPingToApi(status_ref, test_button_ref, intention)`
3. 参数: `status_ref` — 状态引用, `test_button_ref` — 按钮引用, `intention` — 操作意图 (测试或考试)。
4. 算法说明: 预处理输入数据, 发送 API 请求, 并处理响应。

#### 3.6.7.25 process\_userinput

1. 功能: 处理用户输入。
2. 格式: `function process_userinput(row_table_DOM_node, id)`
3. 参数: `row_table_DOM_node` — 行节点, `id` — 行 ID。
4. 算法说明: 将用户输入添加到集合中。

#### 3.6.7.26 addBar

1. 功能: 添加新输入行。
2. 格式: `function addBar(e)`
3. 参数: `e` — 事件对象。
4. 算法说明: 阻止默认事件, 添加新输入行。

#### 3.6.7.27 setInputVal

1. 功能: 设置输入值。
2. 格式: `function setInputVal(value)`
3. 参数: `value` — 输入值。
4. 返回值: 无。
5. 算法说明: 更新 `input_val` 变量。

#### 3.6.7.28 image\_click\_handler

1. 功能: 处理图像点击事件。
2. 格式: `function image_click_handler(event)`
3. 参数: `event` — 事件对象。
4. 算法说明: 更新行条目数组, 添加新行。

#### 3.6.7.29 WarningSign

1. 功能: 显示警告标志。
2. 格式: `function WarningSign()`
3. 返回值: 返回警告标志组件。
4. 算法说明: 返回包含警告信息的 React 组件。

### 3.6.7.30 set\_text\_form

1. 功能: 设置文本表单值。
2. 格式: `function set_text_form(event)`
3. 参数: `event` — 事件对象。
4. 算法说明: 更新 `input_val` 变量。
- 5.

### 3.6.7.31 Visual

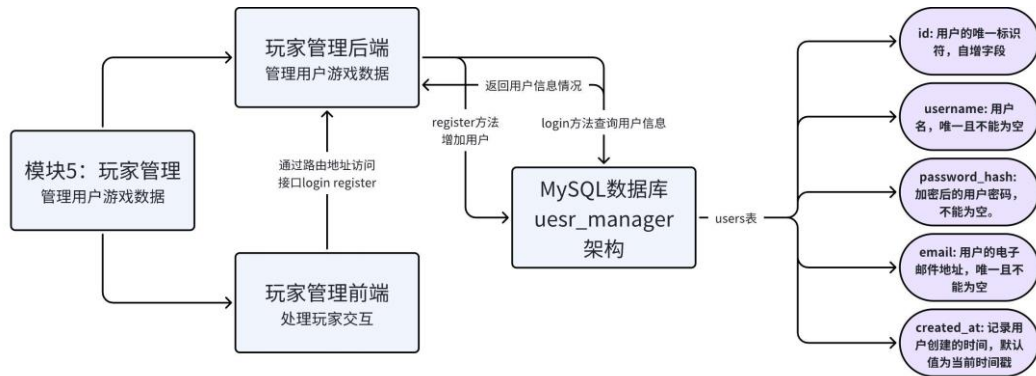
1. 功能: `Visual` 是一个 React 函数组件, 用于显示和管理有限状态自动机 (Finite State Automaton, FSA) 的图形表示。用户可以添加节点和边, 设置初始状态和接受状态, 编辑边的标签, 并通过交互操作修改图的结构。
2. 参数: `Visual` 函数组件没有直接的参数传入, 但使用了 React 的状态管理和上下文来管理和共享组件的数据。
3. 全局变量
  - `img_index`: 图像数组索引。
  - `img_array`: 包含不同状态下的图片路径数组。
  - `node_id_global`: 全局节点 ID 计数器。
  - `img_bar_status_did_mount`: 记录图片状态栏是否已挂载。
  - `nodesDS`: vis.js 的数据集对象, 包含节点数据。
  - `edgesDS`: vis.js 的数据集对象, 包含边数据。
  - `graph`: 包含节点和边的数据对象。
  - `in_initial_mode`: 初始模式标志。
  - `in_accepting_mode_`: 接受模式标志。
  - `delete_lock`: 删除锁定标志。
  - `inputVal`: 输入值。
  - `edgeDeletion`: 边删除标志。
4. 局部变量
  - `show`: 状态变量, 控制模态框的显示与隐藏。
  - `warning`: 状态变量, 控制警告信息的显示与隐藏。

- `master_context`: 使用 `AutomataContext` 上下文，包含全局图表对象和数据集。
  - `wrapper`: `useRef` 钩子，引用图表容器的 DOM 节点。
  - `img_status`: `useRef` 钩子，引用图表状态图像的 DOM 节点。
  - `network`: 图表网络对象。
5. 返回值: `Visual` 函数组件没有显式的返回值，它返回的是组件的 UI 结构。
6. 算法说明:
- a. 初始化网络和事件监听器:
    - 使用 `useEffect` 钩子在组件挂载时初始化 `vis.Network`，并设置各种事件监听器。
    - 初始化图表的节点和边数据集。
  - b. 事件处理:
    - `emptyCanvasClickHandler`: 处理空画布点击事件，添加新节点。
    - `deleteNodeWithCtrl`: 使用 `Ctrl` 键删除选中的节点。
    - `handleShiftClick`: 处理 `Shift` 键按下事件，启用编辑边模式。
    - `deselectAllModes`: 取消所有模式的选择。
    - `nodesOfEdgeId`: 获取边的起始和终止节点标签。
    - `ChangeEdgeText`: 更改边的文本标签，进行输入校验并更新数据集。
    - `findEdgeByNodes`: 根据起始和终止节点查找边的 ID。
    - `toEditEdgeMode`: 启用编辑边模式。
    - `toAddNodeMode`: 启用添加节点模式。
    - `setInitial`: 设置初始状态模式。
    - `setAccepting`: 设置接受状态模式。
    - `newNodeLabel`: 生成不重复的新节点标签。
    - `populateNode`: 添加新节点并随机定位。
    - `deleteNodeOrEdge`: 删除选中的节点或边。
7. 使用约束
- 组件的生命周期: 需要在组件的整个生命周期中管理事件监听器和全局状态，避免内存泄漏。

- 图形操作的有效性：确保操作的有效性，如检查节点或边是否存在。
- 动态内容管理：保持状态同步，避免出现不一致的用户界面状态。
- 用户交互：用户输入需进行校验，确保输入的合法性和一致性。

### 3.7、模块 5：玩家管理

#### 3.7.1、设计图



#### 3.7.2、功能描述

玩家管理模块的业务功能：

- 注册用户：处理新用户的注册请求，将用户信息保存到数据库中，并加密用户密码。
- 登录用户：处理用户登录请求，验证用户信息并根据验证结果进行相应处理（成功时重定向到游戏页面，失败时返回错误消息）。

#### 3.7.3、输入数据

##### 3.7.3.1 注册模块

- 输入数据：
  - `username`：用户名，必填项，非空。
  - `password`：密码，必填项，非空，至少 6 个字符。
  - `email`：电子邮件，必填项，格式验证。
- 有效性检验规则：
  - 用户名不能为空，长度限制。
  - 密码不能为空，长度限制（至少 6 个字符）。
  - 邮箱格式验证。



- 数据来源：前端表单输入。

### 3.7.3.2 登录模块

- 输入数据：
  - `username`：用户名，必填项，非空。
  - `password`：密码，必填项，非空。
- 有效性检验规则：
  - 用户名不能为空。
  - 密码不能为空。
- 数据来源：前端表单输入。

### 3.7.4、输出数据

#### 3.7.4.1 注册模块

- 输出数据：
  - 成功消息：`{"message": "User registered successfully"}`。
  - 错误消息：`{"error": "Database error"}` 或 `{"error": "Unexpected error"}`。

#### 3.7.4.2 登录模块

- 输出数据：
  - 成功消息：`{"redirect": "https://wangdianyun.github.io/GodSay/"}`。
  - 错误消息：`{"error": "Invalid username or password"}`, `{"error": "Database error"}` 或 `{"error": "Unexpected error"}`。

### 3.7.5、数据设计

#### 3.7.5.1 数据库表：`users`

- 字段：
  - `username`：字符串，主键，用户名。
  - `password_hash`：字符串，加密后的密码。
  - `email`：字符串，用户邮箱。

#### 3.7.5.2 局部数据结构：

- `db_config`：包含数据库连接信息的字典。

- `data`: 包含用户提交的注册或登录信息的字典。
- `user`: 包含从数据库中查询到的用户信息的字典。

### 3.7.6、算法和流程

#### 3.7.6.1 注册用户流程:

- a. 接收前端发送的注册请求 (POST) 。
- b. 从请求中提取用户信息 (用户名、密码、邮箱) 。
- c. 验证输入数据的有效性。
- d. 加密用户密码。
- e. 连接数据库, 插入用户信息。
- f. 返回注册成功消息或错误消息。

#### 3.7.6.2 登录用户流程:

- a. 接收前端发送的登录请求 (POST) 。
- b. 从请求中提取用户信息 (用户名、密码) 。
- c. 连接数据库, 查询用户信息。
- d. 验证用户密码。
- e. 返回登录成功消息 (重定向到游戏页面) 或错误消息。

### 3.7.7、函数说明

#### 3.7.7.1 函数 `register`

- 文件: `app.py`
- 功能: 处理用户注册请求, 插入用户信息到数据库。
- 格式: `def register():`
- 参数: 无。
- 全局变量: `db_config`。
- 局部变量: `data`, `username`, `password`, `email`, `password_hash`, `connection`, `cursor`。
- 返回值: `json` 格式的响应消息。
- 算法说明: 见 3.6.6 注册用户流程。
- 使用约束: POST 请求, `Content-Type` 为 `application/json`。

### 3.7.7.2 函数 login

- 文件: `app.py`
- 功能: 处理用户登录请求, 验证用户信息。
- 格式: `def login():`
- 参数: 无。
- 全局变量: `db_config`。
- 局部变量: `data, username, password, connection, cursor, user`。
- 返回值: `json` 格式的响应消息。
- 算法说明: 见 3.6.6 登录用户流程。
- 使用约束: POST 请求, `Content-Type` 为 `application/json`。

### 3.7.8 全局数据结构与该模块的关系

该模块访问了全局数据结构 `db_config`, 用于连接数据库。

## 4、接口设计

### 4.1、用户接口

用户可以使用浏览器访问 `https://wangdianyun.github.io/GodSay/`, 通过 HTTP 或 HTTPS 协议获取服务器响应, 得到游戏页面, 并在浏览器中进行游戏。

### 4.2、外部接口

本软件使用了 ChatGPT API 与 RFLAP API 外部接口, 通过向远程服务器提交 POST 请求以通信。

#### 4.2.1 OpenAI Chat Completion API 接口:

- 请求方式: 通过向远程服务器提交附带请求数据的 POST 请求以通信, 用于生成对话的回复;
- URL: `https://api.openai.com/v1/chat/completions`;
- 接口的接入方式:
  - i. 准备请求消息:
    - `messages` 数组中包含了多个对话条目, 构建了一个对话上下文。包括一个系统消息 (`system`), 描述世界设定和当前剧情, 用户消息 (`user`), 以及从对话历史 (`dialogHistory`) 中提取的对话条目。

- ii. 请求负载 (Payload) :
  - `payload` 是一个包含模型名称 (`model`) 和消息 (`messages`) 的对象。
- iii. 发送请求:
  - 使用 `fetch` 发送 POST 请求到 OpenAI 的聊天完成端点。
  - 请求头 (`headers`) 中设置了内容类型为 `application/json`, 并包含了认证所需的 `Authorization` 字段 (携带 API 密钥 `apiKey`) 。
- iv. 处理响应:
  - 响应解析为 JSON 格式的数据。
  - 从响应中提取生成的回复 (如果有), 并将其添加到对话历史 (`dialogHistory`) 中。
- v. 处理对话:
  - 将相应得到的回复储存到游戏变量中。
- 接口的详细定义:

```
async function sendMessage(message,currentPlot){
  Graphics.stopGameLoop();
  plot = plot currentPlot
  let url = "https://api.openai.com/v1/chat/completions";
  let messages = [
    {
      "role": "system",
      "content": `World setting: ${worldSetting}. Current plot: ${plot}.`
    },
    ...dialogHistory.map(entry => ({
      "role": entry.speaker === "Player" ? "user" : "assistant",
      "content": entry.message
    })),
    {
      "role": "user",
      "content": message
    }
  ];

  let payload = {
    model: modelName,
    messages: messages
  };
}
```

```

let response = await fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ' + apiKey
  },
  body: JSON.stringify(payload)
});
let data = await response.json();
dialogHistory.push({speaker: "Player", message: message});
if (data.choices && data.choices.length 0) {
  let botResponse = data.choices[0].message.content.trim();
  dialogHistory.push({speaker: "ChatGPT", message: botResponse});
  Graphics.startGameLoop();
  return botResponse;
}
Graphics.startGameLoop();
return "No response from ChatGPT.";
}

```

#### 4.2.2 RFLAP API 接口：

- 请求方式：根据条件 `master_context.PDA` 决定请求 URL，然后使用构造好的 json 向指定的 URL 发起 POST 请求，用于判断用户构造的自动机是否能满足要求。
- URL: `https://rflap.acmuic.app/pda` 或 `https://rflap.acmuic.app/automata`
- 接口的接入方式：
  - i. 准备 URL
    - 根据条件 `master_context.PDA` 决定的，分别指向不同的 Rust API 端点。
  - ii. 数据包处理
    - 如果 `packet_to_misha_the_microsoft_engineer.PDA` 为真，需要对数据包进行处理，删除特定的字段并使数据包标准化。
  - iii. 构建请求对象
    - 构建一个 POST 请求对象，包括请求方法、跨域模式、请求头和请求体（包含 JSON 格式的数据包）。
  - iv. 发送请求

- 使用 `fetch` 发送请求，并捕获可能的异常。

#### v. 处理响应与错误处理

#### ◦ 接口的详细实现

```

• async function postToRustApi(status_ref, test_button_ref) {
  let endpoint = master_context.PDA ? "pda" : "automata";
  /* for testing:
  let url = `${window.location.origin}/` + endpoint;
  */
  let url = "https://rflap.acmuic.app/" + endpoint;

  if (packet_to_misha_the_microsoft_engineer.PDA) {
    delete packet_to_misha_the_microsoft_engineer.state_names;
    delete packet_to_misha_the_microsoft_engineer.determinism;
    delete packet_to_misha_the_microsoft_engineer.alphabet;
    delete packet_to_misha_the_microsoft_engineer.PDA;
    packet_to_misha_the_microsoft_engineer.transition_function.forEach(
      (ar_, id) => {
        ar_.forEach((item, index) => {
          if (item == "ε") {
            ar_[index] = "!";
          }
        });
      }
    );
    packet_to_misha_the_microsoft_engineer.stack_alphabet.map(
      (item, index) => {
        if (item == "ε") {
          packet_to_misha_the_microsoft_engineer.stack_alphabet[index] = "!";
        }
      }
    );
    packet_to_misha_the_microsoft_engineer.transition_alphabet.map(
      (item, index) => {
        if (item == "ε") {
          packet_to_misha_the_microsoft_engineer.transition_alphabet[index] =
            "!";
        }
      }
    );
  }
  //

  let postingObject = {

```

```

method: "POST",
mode: "cors",
// cache:"no-cache",
// credentials: "same-origin",
headers: {
  "Content-Type": "application/json",
},
// redirect:"follow",
// referrer: "no-referrer",
body: JSON.stringify(packet_to_misha_the_microsoft_engineer),
};

const pad = (n, width, z) => {
  z = z || "0";
  n = n + "";
  return n.length = width
    ? n
    : new Array(width - n.length + 1).join(z) + n;
};

const toMins = (seconds) =>
  Math.floor(seconds / 60).toString() +
  ":" +
  pad(Math.round(seconds % 60), 2).toString();

const getMinsIntoSession = (sessionStart, sessionPing) =>
  toMins((sessionPing - sessionStart) / 1000);

testID = uuidv4();

const createTestDotnet = (testID) => {
  const getNumAccepting = (statesMap) =>
    Object.values(statesMap).reduce((t, bool) => t + (bool ? 1 : 0), 0);
  const getMode = () => {
    if (master_context.PDA) {
      return "PDA";
    } else {
      let map = {
        "Non-Deterministic Finite Automata": "NFA",
        "Deterministic Finite Automata": "DFA",
        "Push-down Automata": "PDA",
      };
      return map[master_context.mode.trim()];
    }
  };
};

return {
  sessionID: master_context.session,
  startTime: master_context.date,
  testTime: getMinsIntoSession(master_context.date, new Date()),
  rustPacket: JSON.stringify(packet_to_misha_the_microsoft_engineer),
};

```

```

mode: getMode(),
initialState: packet_to_misha_the_microsoft_engineer.start_state,
numStates: Object.keys(packet_to_misha_the_microsoft_engineer.states)
    .length,
numAccepting: getNumAccepting(
    packet_to_misha_the_microsoft_engineer.states
),
numTransitions:
    packet_to_misha_the_microsoft_engineer.transition_function.length,
testStrings: JSON.stringify(
    packet_to_misha_the_microsoft_engineer.input_strings
),
testID: testID,
};
};
//Check for Errors via the error_object
if (error_object.multiple_initial_states) {
    alert("\tMultiple Initial States!");
    //reset object for next API request
    error_object = {
        no_label_transition: false,
        multiple_initial_states: false,
        no_initial_state: false,
    };

    return null;
}
if (error_object.no_initial_state) {
    alert("请设置初始状态");
    error_object = {
        multiple_initial_states: false,
        no_label_transition: false,
        no_initial_state: false,
        epsilon_on_DFA: false,
        no_label_on_dfa: false,
    };
    animateIntoNeutral(status_ref, test_button_ref);
    return null;
}
let Algorithms_are_the_computational_content_of_proofs;
try {
    Algorithms_are_the_computational_content_of_proofs = await fetch(
        url,
        postingObject
    );
}

```



```

} catch (e) {}

//reset error_object
error_object = {
  multiple_initial_states: false,
  no_label_transition: false,
  no_initial_state: false,
  epsilon_on_DFA: false,
};
return await Algorithms_are_the_computational_content_of_proofs.json();
}

```

## 4.3、内部接口

### 4.3.1、接口说明

#### 4.3.1.1 postMessage 接口

FLAP 子模块在游戏主体中被调用时，当测试样例被玩家构造的自动机（文法、表达式）全部接受后，FLAP 子模块通过 `postMessage` 接口向游戏主体发送信号，游戏主体关闭页面并继续剧情。

`postMessage` 接口定义：

接受参数：

`result`: 玩家在点击镌刻按键后的测试结果 布尔值

#### 4.3.1.2 WebWindow 接口

游戏主体子模块通过 `WebWindow` 接口向 FLAP 子模块发起请求，FLAP 子模块返回自动机解题页面供玩家完成任务，并返回任务完成情况。当任务完成后，FLAP 子模块向游戏主体发送信号，游戏主体关闭页面并继续剧情。

`WebWindow` 接口定义：

接受参数：

`URL`: 需要在游戏中访问的页面 字符串类型

`exam_data`: 本轮任务中需要玩家构造的自动机（或文法、表达式）可接受的字符串样例 字符串数组

`mode`: 本轮任务的类型 字符串类型 可选值: `nfa dfa cfg reg pda`

`x`: 页面中心点在游戏游戏中的 x 坐标位置 整型

`y`: 页面中心点在游戏游戏中的 y 坐标位置 整型

width: 页面在游戏时的宽度 整型  
height: 页面在游戏时的高度 整型  
scrolling: 页面是否展示滚动条 布尔值  
center: 页面是否居中, 居中则忽略 x, y 取值 布尔值  
switchId: 完成任务后改变的游戏主体开关 ID 整型

#### 4.3.1.3 Login 接口

登录页面通过 Login 接口向玩家管理子模块发起请求, 玩家管理子模块返回玩家信息, Login 接口再接受玩家信息并重定向至游戏主体。

接受参数:

username: 用户输入的账户名 明文字符串  
password: 用户输入的密码 字符串

#### 4.3.1.4 Register 接口

登录页面通过 Register 接口向玩家管理子模块发起请求, 玩家管理子模块接受新增用户注册请求并修改用户表。

接受参数:

username: 用户输入的账户名 明文字符串  
password: 用户输入的密码 字符串  
email: 用户的邮箱

### 4.3.2、调用方式

#### 4.3.1.1 postMessage 接口

```
window.top.postMessage(result, '*');
```

#### 4.3.1.2 WebWindow 接口

```
var webWindow = new WebWindow('https://wangdianyun.github.io/GodSay-plugin-automata', ['((ccc)(aa))'], 'nfa', 0, 0, 800, 600, false, true, 80)
```

#### 4.3.1.3 Login 接口

```
<button onclick="login()">登录游戏</button>
```

## 5、数据库设计

### 5.1 MySQL 用户信息表：

- `id`: 用户的唯一标识符，自增字段。
- `username`: 用户名，唯一且不能为空。
- `password_hash`: 加密或散列后的用户密码，不能为空。
- `email`: 用户的电子邮件地址，唯一且不能为空。
- `created_at`: 记录用户创建的时间，默认值为当前时间戳。

### 5.2 JSON 游戏数据库：

- `Actors`: 用于储存玩家信息
  - 格式实例:

```
{
  "id": 1,
  "battlerName": "main_actor_fight",
  "characterIndex": 0,
  "characterName": "main_actor",
  "classId": 1,
  "equips": [
    0,
    0,
    0,
    0,
    0
  ],
  "faceIndex": 0,
  "c": "main_actor",
  "traits": [],
  "initialLevel": 1,
```

```
"maxLevel": 99,  
"name": "玩家",  
"nickname": "",  
"note": "",  
"profile": ""  
},
```

- **Animations**: 用于储存动画信息
  - 格式实例:

```
{  
  "id": 1,  
  "displayType": 0,  
  "effectName": "HitPhysical",  
  "flashTimings": [  
    {  
      "frame": 0,  
      "duration": 30,  
      "color": [  
        255,  
        0,  
        0,  
        255  
      ]  
    }  
  ],  
  "name": "Enemy Damage",  
  "offsetX": 0,  
  "offsetY": 0,  
  "rotation": {  
    "x": 0,  
    "y": 0,  
    "z": 0
```

```

},
"scale": 10,
"soundTimings": [
  {
    "frame": 0,
    "se": {
      "name": "Damage1",
      "pan": 0,
      "pitch": 100,
      "volume": 90
    }
  }
],
"speed": 200,
"timings": []
}

```

- **Armors**: 用于储存游戏装备信息
  - 格式实例:

```

{
  "id": 1,
  "atypeld": 1,
  "description": "[General Equipment] A vest made of tanned hide.",
  "etypeld": 4,
  "traits": [
    {
      "code": 22,
      "dataId": 1,
      "value": 0
    }
  ],
  "iconIndex": 135,

```

```
"name": "Leather Vest",
"note": "",
"params": [
  0,
  0,
  0,
  3,
  0,
  0,
  0,
  0
],
"price": 200
},
```

- **Classes**: 用于储存游戏职业信息

- 格式实例:

```
{
  "id": 1,
  "expParams": [
    30,
    20,
    25,
    30
  ],
  "traits": [
    {
      "code": 23,
      "dataId": 0,
      "value": 1
    }, "code": 22,
    "dataId": 0,
```

```
"value": 1
},
{
  "code": 22,
  "dataId": 1,
  "value": 0.05
},
{
  "code": 22,
  "dataId": 2,
  "value": 0.1
},
{
  "code": 41,
  "dataId": 2,
  "value": 1
},
{
  "code": 51,
  "dataId": 2,
  "value": 0
},
{
  "code": 51,
  "dataId": 1,
  "value": 1
},
{
  "code": 52,
  "dataId": 1,
  "value": 0
},
```

```

{
  "code": 52,
  "dataId": 3,
  "value": 0
},
{
  "code": 52,
  "dataId": 5,
  "value": 0
},
{
  "code": 41,
  "dataId": 1,
  "value": 1
}
],
"learnings": [],
"name": "Swordsman",
"note": "",
{
  "params": [升级信息]
}

```

- **CommonEvents**: 用于储存一般事件信息
  - 格式实例:

过长，不适合展示

- **Enemies**: 用于储存敌对单位信息
  - 格式实例:

```

{
  "id": 1,
  "actions": [
    {

```



```
    "conditionParam1": 0,
    "conditionParam2": 0,
    "conditionType": 0,
    "rating": 5,
    "skillId": 1
  }
],
"battlerHue": 0,
"battlerName": "",
"droplItems": [
  {
    "dataId": 1,
    "denominator": 1,
    "kind": 0
  },
  {
    "dataId": 1,
    "denominator": 1,
    "kind": 0
  },
  {
    "dataId": 1,
    "denominator": 1,
    "kind": 0
  }
],
"exp": 10,
"traits": [
  {
    "code": 22,
    "dataId": 0,
    "value": 0.95
  }
]
```

```
    },  
    {  
      "code": 22,  
      "dataId": 1,  
      "value": 0.05  
    },  
    {  
      "code": 31,  
      "dataId": 1,  
      "value": 0  
    }  
  ],  
  "gold": 5,  
  "name": "wolf",  
  "note": "",  
  "params": [  
    200,  
    0,  
    25,  
    20,  
    20,  
    20,  
    10,  
    10  
  ]  
}
```

- **Items**: 用于储存游戏道具信息

- 格式实例:

```
{  
  "id": 1,  
  "animationId": 0,
```

```
"consumable": true,
"damage": {
  "critical": false,
  "elementId": 0,
  "formula": "",
  "type": 0,
  "variance": 0
},
"description": "Restores HP by 500 points.",
"effects": [
  {
    "code": 11,
    "dataId": 0,
    "value1": 0,
    "value2": 500
  }
],
"hitType": 0,
"iconIndex": 176,
"itypeld": 1,
"name": "Potion",
"note": "<action: Recovery item>",
"occasion": 0,
"price": 100,
"repeats": 1,
"scope": 7,
"speed": 0,
"successRate": 100,
"tpGain": 0
},
```

- Map: 用于储存游戏地图信息
  - 格式实例:

过长不适合展示

- **Skills**: 用于储存游戏技能信息
  - 格式实例:

```
{
  "id": 1,
  "animationId": -1,
  "damage": {
    "critical": true,
    "elementId": -1,
    "formula": "a.atk * 4 - b.def * 2",
    "type": 1,
    "variance": 20
  },
  "description": "",
  "effects": [
    {
      "code": 21,
      "dataId": 0,
      "value1": 1,
      "value2": 0
    }
  ],
  "hitType": 1,
  "iconIndex": 76,
  "message1": "%1 attacks!",
  "message2": "",
  "mpCost": 0,
  "name": "Attack",
  "note": "Skill #1 corresponds to the attack command.\n<action: Normal  
attack>\n<noAttackFrame: 10>.",
  "occasion": 1,
```

```
"repeats": 1,
"requiredWtypeId1": 0,
"requiredWtypeId2": 0,
"scope": 1,
"speed": 0,
"stypId": 0,
"successRate": 100,
"tpCost": 0,
"tpGain": 5,
"messageType": 1
}
```

- **States**: 用于储存玩家特殊状态信息
  - 格式实例:

```
{
  "id": 1,
  "autoRemovalTiming": 0,
  "chanceByDamage": 100,
  "iconIndex": 1,
  "maxTurns": 1,
  "message1": "%1 has fallen!",
  "message2": "%1 is slain!",
  "message3": "",
  "message4": "%1 revives!",
  "minTurns": 1,
  "motion": 3,
  "name": "Dead",
  "note": "State #1 will be added when HP hits 0.",
  "overlay": 0,
  "priority": 100,
  "releaseByDamage": false,
  "removeAtBattleEnd": false,
```

```

"removeByDamage": false,
"removeByRestriction": false,
"removeByWalking": false,
"restriction": 4,
"stepsToRemove": 100,
"traits": [
  {
    "code": 23,
    "dataId": 9,
    "value": 0
  }
],
"messageType": 1
},

```

- **System**: 用于定义提示术语等定制化信息

- 格式实例:

过长不适合展示

- **Tilesets**: 用于定义游戏局内视觉材质

- 格式实例:

过长不适合展示

- **Troops**: 用于定义游戏载具

- 格式实例:

```

[
  null,
  {
    "id": 1,
    "members": [],
    "name": "",
    "pages": [

```

```

{
  "conditions": {
    "actorHp": 50,
    "actorId": 1,
    "actorValid": false,
    "enemyHp": 50,
    "enemyIndex": 0,
    "enemyValid": false,
    "switchId": 1,
    "switchValid": false,
    "turnA": 0,
    "turnB": 0,
    "turnEnding": false,
    "turnValid": false
  },
  "list": [
    {
      "code": 0,
      "indent": 0,
      "parameters": []
    }
  ],
  "span": 0
}
]
}
]

```

- **Weapons**: 用于定义游戏武器
  - 格式实例:

```

{
  "id": 1,

```

```
"animationId": 6,
"description": "神的礼物指引你的力量走向正途...",
"etypeld": 1,
"traits": [
  {
    "code": 31,
    "dataId": 1,
    "value": 0
  },
  {
    "code": 22,
    "dataId": 0,
    "value": 0
  }
],
"iconIndex": 97,
"name": "晶匕",
"note": "",
"params": [
  0,
  0,
  8,
  0,
  0,
  0,
  0,
  0
],
"price": 300,
"wtypeld": 2
}
```



## 6、系统出错处理

### 6.1、出错信息

- ARPG\_WeaponAnimation.js 文件中：

- 错误信息：动态加载模块错误
  - 描述：在模块动态加载时出现错误。

```
throw new Error('Dynamic require of "' + x + '" is not supported');
```

- 错误信息：方向错误
  - 描述：当用户方向不在预期范围内时触发。

```
throw new Error(`${this.user().direction()} is not found.`);
```

- CharacterCollisionEx.js 文件中：

- 错误信息：参数转换错误
  - 描述：在插件参数解析时，如果参数类型不匹配，会出现此错误。

```
throw new Error(`Invalid param: ${param}`);
```

- 错误信息：无限循环错误
  - 描述：在参数展开过程中检测到无限循环。

```
throw new Error("endless loop error");
```

- 错误信息：无效数组类型
  - 描述：在参数转换过程中，如果数组类型无效，会出现此错误。

```
throw new Error(`Invalid array type: ${type}`);
```

- 错误信息：无效对象类型
  - 描述：在参数转换过程中，如果对象类型无效，会出现此错误。

```
throw new Error(`Invalid object type: ${type}`);
```

- 错误信息：未知类型

- 描述：在参数转换过程中，如果遇到未知类型，会出现此错误。

```
throw new Error(`Unknow type: ${type}`);
```

- 错误信息：无效的角色规格

- 描述：在查找角色时，如果角色规格无效，会出现此错误。

```
throw new Error(`${JSON.stringify(param)} is invalid.`);
```

- 错误信息：事件名称未找到

- 描述：在查找事件时，如果事件名称未找到，会出现此错误。

```
throw new Error(`Event name(${eventName}) is not found.`);
```

## 6.2、补救措施

- ARPG\_WeaponAnimation.js

- 动态加载模块错误：确保在打包和部署时，所有必要的模块都包含在内。避免在浏览器环境中使用 `require` 函数。
- 方向错误：确保角色方向在预期范围内（如 2、4、6、8）。可以在输入处理逻辑中添加检查，防止传递无效的方向值。

- CharacterCollisionEx.js

- 参数转换错误：确保插件参数的类型和格式正确。可以在参数定义和初始化时进行验证。
- 无限循环错误：检查参数展开逻辑，确保不会进入无限循环。可以通过设置最大递归深度来防止此类错误。
- 无效数组类型：确保在参数转换过程中，数组类型定义正确。可以在参数解析时添加类型检查。
- 无效对象类型：确保在参数转换过程中，对象类型定义正确。可以在参数解析时添加类型检查。
- 未知类型：在参数转换过程中，确保所有类型都被正确处理。可以在类型转换逻辑中添加更多的类型支持。
- 无效的角色规格：确保角色规格参数正确。可以在传递参数前进行验证，避免传递无效的参数。

- 事件名称未找到：确保事件名称在地图事件中存在。可以在传递事件名称前进行检查，确保名称有效。

## 7、其他设计

无，设计信息均在文档上文中有所体现。